



Discrete optimization

Combinatorial Benders decomposition for single machine scheduling in additive manufacturing with two-dimensional packing constraints

Zhaofang Mao^{a,b}, Enyuan Fu^a, Dian Huang^{a,c}, Kan Fang^{a,b,*}, Lin Chen^d^a College of Management and Economics, Tianjin University, Tianjin 300072, China^b Laboratory of Computation and Analytics of Complex Management Systems (CACMS), Tianjin University, Tianjin 300072, China^c School of New Media and Communication, Tianjin University, Tianjin 300072, China^d College of Computer Science, Zhejiang University, Hangzhou 310027, China

ARTICLE INFO

Keywords:

Scheduling

Additive manufacturing

Two-dimensional packing

Approximation algorithm

Combinatorial benders decomposition

ABSTRACT

This paper addresses a single-machine scheduling problem in additive manufacturing (AM). We focus on the direct metal laser sintering (DMLS) technology of an AM process, where parts can be produced simultaneously in a batch. In this problem, we consider simultaneously the assignment of parts to batches and the packing of parts onto two-dimensional surfaces of the build chamber to minimize the makespan objective. To solve this problem, a mixed integer linear programming model is formulated, and an approximation algorithm is proposed, followed by an exact algorithm based on combinatorial Benders decomposition method with various Benders cuts and acceleration strategies (Algorithm CBD). Finally, we conduct a comprehensive computational study to verify the efficiency of our proposed Algorithm CBD. The computational results show that our proposed Algorithm CBD can effectively solve the test instances with different sizes.

1. Introduction

Additive Manufacturing (AM), commonly known as 3D printing, is an innovative manufacturing technology that uses 3D digital model files to create objects layer-by-layer (Gardan, 2016), which can dramatically shorten lead time, reduce material loss, create complex geometries without molds, and thus fix some technical issues that are difficult to deal with in traditional manufacturing processes (Westerweel et al., 2018). With the rapid development of AM technology, it has become an important part of the fourth industrial revolution and has been widely used in various manufacturing industries (Attaran, 2017). According to a report by Grand View Research (2022), the global AM market size was valued at \$13.84 billion in 2021 and is expected to increase at a 20.8% compound annual growth rate from 2022 to 2030.

Although additive manufacturing has many advantages such as providing personalized customization and on-demand products with less material loss, it also has relatively low production efficiency compared with traditional manufacturing. In recent years, researchers have found that the processing efficiency in additive manufacturing can be considerably improved upon by scheduling different parts appropriately on AM machines and thus reduce the total processing cost (Li et al., 2017). In particular, several researchers have noticed that for some AM technologies, multiple parts can be processed simultaneously in a

batch to further reduce the build time of parts and save on production costs. For example, Fig. 1 shows the schematic of the direct metal laser sintering (DMLS) technology of the AM process, which produces multiple parts simultaneously by using a high-power laser beam to densify the metal powder layer by layer. After processing one layer, the powder delivery platform rises one layer and the build platform descends one layer (with a given layer thickness). Then the roller scrapes the powder from the powder delivery platform onto the build platform. The laser densifies the new metal powder and fuses it with the previous layer. The above processes of powder layering and laser melting alternate with each other until all the allocated parts in the batch are produced (Altekin & Bukchin, 2022).

In order to verify whether a set of parts can be feasibly packed into a batch, researchers usually modeled it as a one-dimensional knapsack constraint (e.g. Altekin & Bukchin, 2022), that is, they only impose a restriction such that the total base area of all parts in the same batch cannot exceed the base area of the build chamber. However, in real additive manufacturing, it is important to ensure that the *two-dimensional packing constraint* is not violated, that is, the parts within the same batch need to be allocated into the build chamber without overlapping. In particular, it is usually assumed that each part is represented by its minimum rectangular bounding box projected on the

* Corresponding author at: College of Management and Economics, Tianjin University, Tianjin 300072, China.

E-mail addresses: maozhaofang@tju.edu.cn (Z. Mao), fe609333118@tju.edu.cn (E. Fu), huangdian@tju.edu.cn (D. Huang), kfang@tju.edu.cn (K. Fang), chenlin198662@zju.edu.cn (L. Chen).<https://doi.org/10.1016/j.ejor.2024.05.001>

Received 8 August 2023; Accepted 1 May 2024

Available online 3 May 2024

0377-2217/© 2024 Elsevier B.V. All rights reserved.

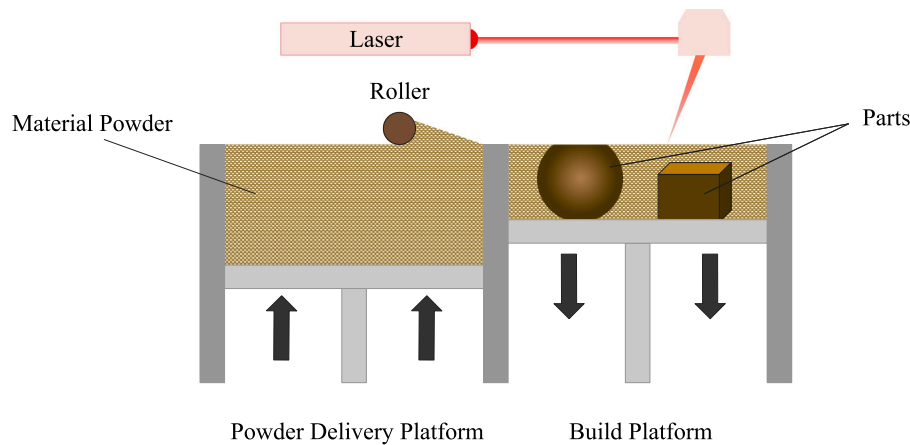


Fig. 1. Schematic for the manufacturing process of DMLS.

XY plane, and all parts need to be packed with their edges parallel to the edges of the build chamber (Che et al., 2021).

The two-dimensional packing constraint is a fundamental requirement in many industrial applications, especially in cutting and packing literature, and various problems with such constraints have been widely investigated, including the two-dimensional orthogonal packing problem (e.g. Clautiaux et al., 2007), two-dimensional bin packing problem (e.g. Lodi et al., 2002), two-dimensional strip packing problem (e.g. Júnior et al., 2022), two-dimensional knapsack problem (e.g. Caprara & Monaci, 2004), and some other problems such as the facility planning or block layout planning problems (e.g. Guo et al., 2023; Heragu, 2022). In particular, according to specific applications, these problems can be further classified into two main variants: the *fixed orientation variant* and the *rotatable variant*. In the fixed orientation variant, all the parts have to be packed in a fixed orientation. For example, articles are not allowed to be rotated when they are paged in newspapers. Conversely, in the rotatable variant, parts are allowed to be rotated by 90° during the packing process. Until recently, both variants have been separately studied by different researchers within specific real-world contexts (e.g. Boschetti & Mingozzi, 2003a, 2003b; Epstein, 2010; Kenmochi et al., 2009).

In this work, we focus on a batch scheduling problem of producing 3D metal parts with fixed orientations on a single additive machine with DMLS technology, in which the machine has a limited capacity and the rectangular bounding box of each part needs to be packed into the machine without overlapping, and the objective is to minimize the makespan. In particular, the processing time of each batch is a weighted sum of the maximum height and total volume of the allocated parts, and the sequence-independent setup time of the machine. For simplicity, we refer to this problem as the *single machine scheduling problem in additive manufacturing with two-dimensional packing constraints*, or the SMSAM-2DP problem for short.

As shown by Kucukkoc (2019), even without the consideration of two-dimensional packing, the corresponding single machine scheduling problem in additive manufacturing is already NP-hard, so the SMSAM-2DP problem is also NP-hard. Since the SMSAM-2DP problem includes a single additive machine, the makespan is only dependent on the number of used batches and the assignment of parts into each batch, while the sequence of batches has no impact on the value of the makespan as all batches must be processed continuously without any idle time. Therefore, in order to achieve the minimum makespan, we only need to simultaneously determine the assignment of parts to the batches and the two-dimensional packing of parts in each batch to ensure the feasibility of the solution.

Over the past decade, there has been an increasing amount of work on various scheduling problems in additive manufacturing. However, most efforts have been focused on investigating scheduling problems

with one-dimensional knapsack constraints. In contrast, only a limited amount of work has been conducted on scheduling problems involving two-dimensional packing constraints. The only four exceptions that we can find are the works by Che et al. (2021), Aloui and Hadj-Hamou (2021), Kucukkoc (2021) and Hu et al. (2022), in which the authors investigated different scheduling problems while taking into the consideration a two-dimensional packing of parts and other technology constraints, and proposed various heuristic-based solution approaches to solve the corresponding problems. To the best of our knowledge, there are no studies that have used exact solution approaches to solve scheduling problems in additive manufacturing with two-dimensional packing. In this work, we aim to fill this gap by developing a combinatorial Benders decomposition algorithm for the SMSAM-2DP problem. To speed up the solution process of our proposed algorithm, we develop various acceleration strategies and Benders cuts, and test its performance via extensive computational experiments.

The rest of this paper is organized as follows. Section 2 reviews the literature that is relevant to this research. Section 3 describes the SMSAM-2DP problem with its associated mixed integer linear programming (MILP) formulation. In Section 4 an approximation algorithm is introduced and its approximation ratio is proved, followed by our proposed combinatorial Benders decomposition-based exact algorithm in Section 5. Section 6 reports the computational experiments on our proposed model and algorithms. Finally, Section 7 concludes the paper and discusses future research directions.

2. Literature review

Since our work focuses on solving the SMSAM-2DP problem with combinatorial Benders decomposition method, we organize our literature review mainly on research papers that consider scheduling problems in additive manufacturing and Benders decomposition methods.

2.1. Relevant literature on scheduling problems in additive manufacturing

As we have mentioned above, the DMLS technology allows the simultaneous processing of multiple parts in a batch, while the processing time of this batch is dependent on several factors including the maximum height and the total volume of the allocated parts, and the setup time of the machine. Therefore, the batch scheduling problem in additive manufacturing is significantly different from the traditional parallel-batch or serial-batch scheduling problems, and is more similar to a recently proposed mixed-batch scheduling problem, which was first proposed by Wang et al. (2020), in which the batch processing time is equal to the weighted sum of the maximum processing time and the total processing time of the allocated parts, and no setup

time is considered. In what follows, we only focus our review on AM scheduling problems in which multiple parts can be allocated into a batch and processed simultaneously. For those works that only allow parts to be processed separately, we refer the reader to a recent survey by Oh et al. (2020).

In recent years, more and more researchers have started to consider various scheduling problems in additive manufacturing with a batch processing mode. Li et al. (2017) initiated the study of scheduling in additive manufacturing based on a batch processing mode, and proposed two different heuristic algorithms to minimize the average production cost per volume of material. Kucukkoc (2019) considered both the scheduling problems of single and multiple parallel AM machines to minimize the makespan, and proposed mixed integer linear programming models for these problems. Zhang et al. (2020) developed an improved evolutionary algorithm to minimize the makespan of the parallel batch scheduling problem in additive manufacturing. Altekin and Bukchin (2022) proposed mixed integer programming formulations for scheduling problems with single and multiple AM machines, where the machines may differ in their size and setting, and both objectives of cost and makespan are considered.

In particular, several researchers have also observed that the two-dimensional packing of parts cannot be simply ignored while solving the scheduling problems in additive manufacturing. Che et al. (2021) studied an unrelated parallel batch processing machine scheduling problem in which the orientation selection and a two-dimensional packing of parts are considered, and the objective is to minimize the makespan. To solve this problem, the authors proposed a simulated annealing algorithm with designed packing strategies and verified its effectiveness by computational studies. Aloui and Hadj-Hamou (2021) investigated the two-dimensional placement and scheduling problem of parts production in additive manufacturing subject to various technology constraints, and proposed a heuristic algorithm to solve large-sized instances. Kucukkoc (2021) investigated the relationship between the two-dimensional packing and scheduling of AM machines under a batch processing mode, and showed that these two problems must be considered simultaneously to ensure the feasibility of solutions. Hu et al. (2022) considered a scheduling problem with unrelated additive machines in parallel subject to two-dimensional packing constraints and unequal part release times, and multiple orientation candidates are available for each part. To minimize the makespan, the authors proposed an adaptive large neighborhood search algorithm to solve large-sized instances.

2.2. Relevant literature on benders decomposition

The SMSAM-2DP problem considered in this paper can be formulated as a mixed integer program that consists of two interrelated problems, i.e., the assignment of parts to batches and a two-dimensional packing problem for each batch. Inspired by this observation, one natural idea to solve the SMSAM-2DP problem is to employ a Benders decomposition method. As we know, Benders decomposition was first proposed by Benders (1962), which can solve various mixed integer programs by decomposing the original problem into a master problem (MP) and subproblem(s) (SP), instead of tackling the entire problem at once. With the implementation of Benders decomposition, various difficult problems can be efficiently solved within reasonable time (Hooker, 2007).

In particular, as a variant of the Benders decomposition, the combinatorial Benders decomposition method, which was first proposed by Hooker (2011), has attracted increasing attention from researchers, as this method can extend itself to successfully solve various mixed integer programming models with special structures (Codato & Fischetti, 2006), such as the classical pickup and delivery problem (e.g. Cortés et al., 2010), bin packing problem (e.g. Côté et al., 2021), scheduling problem (e.g. Fang et al., 2021; Li et al., 2022), and assembly line balancing problem (e.g. Huang et al., 2022). In a combinatorial Benders

decomposition algorithm, we can iteratively exclude the current solution of the master problem from further consideration by generating combinatorial or feasibility cuts through the logic between the master problem and the subproblems, and such process continues until an optimal solution of the original problem has been obtained (Rahmani et al., 2017). However, to the best of our knowledge, there has not been any work yet on solving the SMSAM-2DP problem with exact algorithms. In this work, we aim to fill this gap by proposing a combinatorial Benders decomposition-based exact algorithm to solve the SMSAM-2DP problem.

3. Problem description and mathematical formulation

The SMSAM-2DP problem can be described as follows. There is a set of parts $I = \{1, 2, \dots, n\}$ that need to be built on a single additive machine using the DMLS technology, where multiple parts can be processed simultaneously as a batch. Similar to the work of Kucukkoc (2019) and Aloui and Hadj-Hamou (2021), we consider a fixed orientation variant of the problem by assuming that the build orientation of each part i has been predetermined. In addition, each part has a height h_i , a volume v_i , and its minimum rectangular bounding box has a length ℓ_i and a width w_i . Let L, W, H be the corresponding length, width and height of the build chamber within the additive machine. Obviously, we must have $\ell_i \leq L, w_i \leq W$, and $h_i \leq H$ for $i \in I$, otherwise this part cannot be processed by the given machine.

Meanwhile, due to the layer-by-layer characteristic, the scanning time and the recoating time of a given batch are proportional to the total volume and the maximum height of the allocated parts in this batch, respectively (Kucukkoc, 2019). We denote VT and HT as the scanning time per unit volume and the recoating time per unit height on the additive machine, and ST as the setup time of this machine, which refers to the time required for preparatory operations such as data preparation, material replenishment on the powder material transport platform, and parameter adjustments. In addition, once a batch is started to process parts, it cannot be interrupted and no parts can be added or removed from this batch until its completion. Then, the processing time of each batch is a weighted sum of maximum height and total volume of the allocated parts, and the sequence-independent setup time of the additive machine (Altekin & Bukchin, 2022). We define a schedule as a *feasible schedule* if all of the parts are allocated into a number of batches so that each batch satisfies the two-dimensional packing constraints, that is, there is no overlap between the rectangular bounding boxes of any two parts in the batch. The objective of the SMSAM-2DP is to find a feasible schedule that minimizes the makespan.

3.1. The mixed integer linear programming (MILP) model for the SMSAM-2DP problem

Let $B = \{1, 2, \dots, n\}$ be the set of batches that may be opened. Obviously, a batch $b + 1$ can be opened only if its previous batch b has already been opened. We define the following decision variables:

- z_b is equal to 1 if batch b is opened, and 0 otherwise;
- u_{ib} is equal to 1 if part i is allocated into batch b , and 0 otherwise;
- left_{ijb} is equal to 1 if part i is located left of part j in batch b , and 0 otherwise;
- below_{ijb} is equal to 1 if part i is located behind part j in batch b , and 0 otherwise;
- h_b is the height of batch b ;
- (x_i, y_i) is the coordinates of the lower-left corner of part i ;
- C_b is the completion time of batch b , and C_0 is defined as 0.

Let M be a sufficiently large number, then the SMSAM-2DP problem can be formulated as follows.

$$\min C_{\max} \quad (1a)$$

$$\begin{aligned}
\text{s.t. } \sum_{b \in B} u_{ib} &= 1 & \forall i \in I & \quad (1b) \\
h_i \cdot u_{ib} &\leq h_b & \forall i \in I, b \in B & \quad (1c) \\
x_i + w_i &\leq W + M \cdot (1 - u_{ib}) & \forall i \in I, b \in B & \quad (1d) \\
y_i + \ell_i &\leq L + M \cdot (1 - u_{ib}) & \forall i \in I, b \in B & \quad (1e) \\
\text{left}_{ijb} + \text{left}_{jib} + \text{below}_{ijb} &+ \text{below}_{jib} \geq u_{ib} + u_{jb} - 1 & \forall i, j \in I, i \neq j, b \in B & \quad (1f) \\
x_i + w_i &\leq x_j + M \cdot (1 - \text{left}_{ijb}) & \forall i, j \in I, i \neq j, b \in B & \quad (1g) \\
y_i + \ell_i &\leq y_j + M \cdot (1 - \text{below}_{ijb}) & \forall i, j \in I, i \neq j, b \in B & \quad (1h) \\
\sum_{i \in I} u_{ib} &\leq M \cdot z_b & \forall b \in B & \quad (1i) \\
z_{b+1} &\leq z_b & \forall b \in B & \quad (1j) \\
C_b &\geq C_{b-1} + VT \cdot \sum_{i \in I} v_i \cdot u_{ib} & & \\
&+ HT \cdot h_b + ST \cdot z_b & \forall b \in B & \quad (1k) \\
C_{\max} &\geq C_b & \forall b \in B & \quad (1l) \\
x_i, y_i &\geq 0 & \forall i \in I & \quad (1m) \\
u_{ib}, z_b, \text{left}_{ijb}, \text{below}_{ijb} &\in \{0, 1\} & \forall i, j \in I, i \neq j, b \in B & \quad (1n) \\
h_b, C_b &\geq 0 & \forall b \in B & \quad (1o)
\end{aligned}$$

The objective function (1a) aims to minimize the makespan. Constraints (1b) ensure that each part i must be allocated to exactly one batch. Constraints (1c) define the height of each batch. Constraints (1d)–(1e) ensure that each part cannot be placed outside the machine's build chamber in horizontal or vertical direction. Constraints (1f)–(1h) guarantee that if two parts i and j are allocated to the same batch, they are not allowed to overlap with each other. Constraints (1i) ensure that batch b is opened if at least one part is allocated to this batch. Constraints (1j) ensure that a batch can be opened only if its previous batch has already been opened. Constraints (1k) define the completion time of each batch. Constraints (1l) define the makespan of the problem. Constraints (1m)–(1o) present the ranges of the decision variables.

It should be noted that the above MILP model exhibits several differences from the previous models that incorporate two-dimensional packing constraints. In the studies by Che et al. (2021) and Hu et al. (2022), additional variables and constraints were introduced concerning orientation selection decisions and their problems were investigated in parallel additive machine environments. In the work of Aloui and Hady-Hamou (2021), they considered the problem of minimizing the total lateness on parallel additive machines and provided varied formulations for calculating batch production times under different AM technologies. Moreover, all these efforts mainly focused on developing various meta-heuristic algorithms to address their respective problems. In contrast to these studies, our focus is on examining a fixed orientation variant of the scheduling problem on a single additive machine, with an objective of minimizing the makespan, and developing exact solution approaches for solving the SMSAM-2DP problem.

4. An approximation algorithm for the SMSAM-2DP problem

Obviously, in any optimal schedule of the SMSAM-2DP problem, there must be no inserted idle time between the processing of any two consecutive batches, otherwise we can always obtain a new schedule with smaller makespan. Note that the total processing time of all the batches can be calculated as follows:

$$P = \sum_{b \in B} P_b = VT \cdot \sum_{i \in I} v_i + HT \cdot \sum_{b \in B} h_b + ST \cdot \sum_{b \in B} z_b. \quad (2)$$

It is easy to see that the total scanning time related to the total volume of all the parts will remain the same among all feasible schedules,

therefore, the optimal makespan only depends on the total recoating time related to the height of each batch and the total setup time related to the number of batches that are opened.

Now suppose that σ^* is an optimal schedule of the SMSAM-2DP problem, and the total number of batches opened for this schedule is t . Without loss of generality, we assume that all the batches are sorted in non-increasing order of their heights, that is, $h_1^* \geq h_2^* \geq \dots \geq h_t^*$, where h_k^* is the height of batch k ($k = 1, \dots, t$) in the optimal schedule. Then, we can calculate the makespan of the optimal schedule σ^* , that is, $C_{\max}(\sigma^*) = VT \cdot \sum_{i \in I} v_i + HT \cdot \sum_{k=1}^t h_k^* + ST \cdot t$.

Based on the above observation, we first divide all the parts into three groups as follows:

$$\begin{aligned}
I_1 &= \left\{ i : w_i \leq \frac{1}{2}W \ \& \ \ell_i \leq \frac{1}{2}L \right\}, \\
I_2 &= \left\{ i : w_i > \frac{1}{2}W \right\}, \\
I_3 &= \left\{ i : w_i \leq \frac{1}{2}W \ \& \ \ell_i > \frac{1}{2}L \right\}.
\end{aligned}$$

For each group I_i , we let n_i denote the number of parts in this group for $i = 1, 2, 3$. In addition, we sort the parts in each group in nonincreasing order of their heights. We denote part j_k^i as the k th part in group I_i , that is, $h_{j_1^i} \geq h_{j_2^i} \geq \dots \geq h_{j_{n_i}^i}$. Then, we propose a greedy-based packing algorithm for the SMSAM-2DP problem as in Algorithm 4.1. We refer to this algorithm as **Algorithm GreedyPack**.

Algorithm 4.1 A greedy-based packing algorithm for the SMSAM-2DP problem

```

1: Initialize:  $\tilde{A} \leftarrow 0, \tilde{L} \leftarrow 0, \tilde{W} \leftarrow 0$ .
2: for  $i = 1$  to 3 do
3:   if  $I_i \neq \emptyset$  then
4:     Open new batch so as to pack the parts for each  $I_i$ . Let  $s_i \leftarrow 1$ .
5:   for  $k = 1$  to  $n_i$  do
6:      $\tilde{A} \leftarrow \tilde{A} + w_{j_k^i} \cdot \ell_{j_k^i}$ .
7:     if  $\tilde{A} \leq \frac{1}{2}WL$  then
8:       Put part  $j_k^i$  into current batch.
9:     else
10:      Close current batch. Open new batch and put part  $j_k^i$  into new batch.
11:       $s_i \leftarrow s_i + 1, \tilde{A} \leftarrow w_{j_k^i} \cdot \ell_{j_k^i}$ .
12:   for  $k = 1$  to  $n_2$  do
13:      $\tilde{L} \leftarrow \tilde{L} + \ell_{j_k^2}$ .
14:     if  $\tilde{L} \leq L$  then
15:       Put part  $j_k^2$  into current batch.
16:     else
17:      Close current batch. Open new batch and put part  $j_k^2$  into new batch.
18:       $s_2 \leftarrow s_2 + 1, \tilde{L} \leftarrow \ell_{j_k^2}$ .
19:   for  $k = 1$  to  $n_3$  do
20:      $\tilde{W} \leftarrow \tilde{W} + w_{j_k^3}$ .
21:     if  $\tilde{W} \leq W$  then
22:       Put part  $j_k^3$  into current batch.
23:     else
24:      Close current batch. Open new batch and put part  $j_k^3$  into new batch.
25:       $s_3 \leftarrow s_3 + 1, \tilde{W} \leftarrow w_{j_k^3}$ .

```

Obviously, for groups I_2 and I_3 , the packing method in Algorithm GreedyPack can always generate feasible packing solutions. Now we verify that a feasible packing solution can also be generated for group I_1 . For any batch b that is generated by Algorithm GreedyPack, suppose the set of parts in this batch is $\tilde{I} \in I_1$, we let $\bar{w} = \max_{i \in \tilde{I}} w_i$, $\bar{\ell} = \max_{i \in \tilde{I}} \ell_i$, $\bar{A} = \sum_{i \in \tilde{I}} w_i \ell_i$ and $x_+ = \max(x, 0)$. Then, if these values satisfy the following conditions in Steinberg's Theorem (Steinberg, 1997), then we can always feasibly pack the parts of $\tilde{I}$ into the same batch.

Theorem 1 (Steinberg's Theorem). *If the following inequalities hold:*

$$\bar{w} \leq W, \bar{\ell} \leq L, 2A \leq WL - (2\bar{w} - W)_+(2\bar{\ell} - L)_+$$

then it is possible to pack all the parts in $\tilde{T}$ into the rectangle with width W and length L .

By Algorithm GreedyPack, we know that $w_i \leq \frac{1}{2}W$ and $\ell_i \leq \frac{1}{2}L$ for any part i in batch $\tilde{T} \in I_1$, and $\tilde{A} \leq \frac{1}{2}WL$. Therefore, the inequalities in Steinberg's Theorem must hold, so Algorithm GreedyPack can always generate a feasible packing solution for I_1 . As a result, Algorithm GreedyPack can provide a feasible packing solution to the SMSAM-2DP problem. As we know, the number of batches opened by Algorithm GreedyPack for each I_i is denoted as s_i for $i = 1, 2, 3$. If h_k^i is the height of batch k for I_i , then we can obtain the following results.

Lemma 1. $s_1 \leq 4t$.

Proof. For each part i in I_1 , a new batch needs to be opened to pack this part only when $\tilde{A} + w_i \ell_i > \frac{1}{2}WL$, where $\tilde{A}$ is the total area of parts in the current batch. Therefore, the total area of parts in any two consecutive batches must be at least $\frac{1}{2}WL$, and thus the total area of parts in I_1 is at least $\frac{s_1}{2} \cdot \frac{1}{2}WL = \frac{1}{4}s_1 \cdot WL$. On the other hand, the total area of all the parts is at most $t \cdot WL$. So we must have

$$\frac{1}{4}s_1 \cdot WL \leq t \cdot WL \Rightarrow s_1 \leq 4t,$$

and thus the lemma follows. $\square$

Lemma 2. *For any $k \geq 0$, we have $h_{4k-3}^1 \leq h_k^*$.*

Proof. The claim is obviously true for $k = 1$ as h_1^* must be the largest height among all the parts. Suppose that the batches for I_1 generated by Algorithm GreedyPack is $1, 2, \dots, 4k-5, 4k-4, 4k-3, \dots$, obviously we must have $h_1^1 \geq \dots \geq h_{4k-5}^1 \geq h_{4k-4}^1 \geq h_{4k-3}^1 \geq \dots$. Note that the total area of the first $4k-4$ batches must be at least $2(k-1) \cdot \frac{1}{2}WL = (k-1)WL$, as a result, the parts in the first $4k-4$ batches by Algorithm GreedyPack cannot be fully packed into $k-1$ batches in the optimal schedule. Therefore, for the parts in the first $4k-4$ batches, there must exist at least one part i' that will be packed into a batch between k and t in the optimal schedule. As we know, the height of any part in the first $4k-4$ batches must be greater than or equal to the height of the next batch, i.e., batch $4k-3$, that is, $h_{i'} \geq h_{4k-3}^1$. So we obtain $h_k^* \geq h_{i'} \geq h_{4k-3}^1$. $\square$

With the above result, it is easy to prove the following result.

Lemma 3. $\sum_{k=1}^{s_1} h_k^1 \leq 4 \cdot \sum_{k=1}^t h_k^*$.

Similarly, we can establish the relationship between s_2 (s_3) and t as follows.

Lemma 4. $s_2 \leq 2t$ and $s_3 \leq 2t$.

Proof. We denote the direction for measuring the width of parts in the horizontal direction, and the one for measuring the length of parts in the vertical direction. Since we have $w_i > \frac{1}{2}W$ for each part i in I_2 , we know that any two parts in I_2 can only be packed together in the vertical direction if their total length is not greater than L . Obviously, in the optimal schedule, any two parts in I_2 cannot be packed together in the horizontal direction either. Therefore, the problem of packing all the parts in I_2 with minimum number of bins can be reduced to the classical one-dimensional bin packing problem, and the packing rule used in Algorithm GreedyPack for I_2 is actually the next-fit algorithm. As shown by Boyar et al. (2012), the absolute approximation ratio of the next-fit algorithm is less than 2, therefore we must have $s_2 \leq 2t$.

Using a similar argument as above, we obtain $s_3 \leq 2t$. $\square$

Lemma 5. *For any $k \geq 0$, we have $h_{2k-1}^2 \leq h_k^*$ and $h_{2k-1}^3 \leq h_k^*$.*

Proof. The claim is obviously true for $k = 1$ as h_1^* must be the largest height among all parts. Suppose that the batches for I_2 generated by Algorithm GreedyPack is $1, 2, \dots, 2k-3, 2k-2, 2k-1, \dots$. Obviously we must have $h_1^2 \geq \dots \geq h_{2k-3}^2 \geq h_{2k-2}^2 \geq h_{2k-1}^2 \geq \dots$. Note that the total length of the first $2k-2$ batches must be at least $(k-1) \cdot L$, as a result, the parts in the first $2k-2$ batches generated by Algorithm GreedyPack cannot be fully packed into $k-1$ batches in the optimal schedule. Therefore, for the parts in the first $2k-2$ batches, there must exist at least one part i' that will be packed into a batch between k and t in the optimal schedule. Obviously, the height of any part in the first $2k-2$ batches must be greater than or equal to the height of the next batch, i.e., batch $2k-1$, that is, $h_{i'} \geq h_{2k-1}^2$. So we obtain $h_k^* \geq h_{i'} \geq h_{2k-1}^2$.

Using a similar argument as above, we can also show that $h_{2k-1}^3 \leq h_k^*$. $\square$

With the above results, we can show the following result.

Lemma 6. $\sum_{k=1}^{s_2} h_k^2 \leq 2 \cdot \sum_{k=1}^t h_k^*$, and $\sum_{k=1}^{s_3} h_k^3 \leq 2 \cdot \sum_{k=1}^t h_k^*$.

Combining all the above lemmas, we obtain the following theorem.

Theorem 2. *The approximation ratio of Algorithm GreedyPack is at most 8.*

Proof. We denote σ as the schedule generated by Algorithm GreedyPack, and $C_{\max}(\sigma)$ as the corresponding makespan of this schedule. Then, we have

$$C_{\max}(\sigma) = VT \cdot \sum_{i \in I} v_i + HT \cdot \left(\sum_{k=1}^{s_1} h_k^1 + \sum_{k=1}^{s_2} h_k^2 + \sum_{k=1}^{s_3} h_k^3 \right) + ST \cdot (s_1 + s_2 + s_3) \quad (3)$$

$$\leq VT \cdot \sum_{i \in I} v_i + HT \cdot 8 \sum_{k=1}^t h_k^* + ST \cdot 8t \quad (4)$$

$$\leq 8 \cdot (VT \cdot \sum_{i \in I} v_i + HT \cdot \sum_{k=1}^t h_k^* + ST \cdot t) \quad (5)$$

$$= 8 \cdot C_{\max}(\sigma^*), \quad (6)$$

and thus the theorem follows. $\square$

From the above theorem, we know that Algorithm GreedyPack could provide a feasible solution of the SMSAM-2DP problem with guaranteed approximation ratio. It should be noted that the above ratio is only a worst-case ratio, and it is not clear whether this ratio is tight. To better evaluate its performance on average, we will later implement this algorithm and compare its experimental results with those of other algorithms using the test instances.

5. A combinatorial benders decomposition algorithm for the SMSAM-2DP problem

Due to the nature of the SMSAM-2DP problem, we propose a combinatorial Benders decomposition algorithm to solve this problem, and denote it as Algorithm CBD. To be specific, we decompose the original problem into a master problem of allocating parts into batches and multiple subproblems of determining the existence of feasible packing solutions for the allocated parts in each batch based on the solution of the master problem. If there exists some subproblems that cannot find a feasible packing solution, then we generate the corresponding combinatorial Benders cuts into the master problem, and continue such process until an optimal solution has been found.

5.1. The master problem

In this section, we present a mathematical formulation of the master problem (i.e., model (7)) to determine the allocation of parts to the

different batches, in which the two-dimensional packing constraints are not included, and the objective is to minimize the makespan.

$$[\text{master}] \quad \min \quad C_{\max} \quad (7a)$$

$$\text{s.t.} \quad \text{Constraints (1b)–(1c),}$$

$$\text{Constraints (1i)–(1l)} \quad (7b)$$

$$\sum_{i \in I} w_i \ell_i \cdot u_{ib} \leq W \cdot L \quad \forall b \in B \quad (7c)$$

$$u_{ib}, z_b \in \{0, 1\} \quad \forall i \in I, b \in B \quad (7d)$$

$$h_b, C_b \geq 0 \quad \forall b \in B \quad (7e)$$

The objective (7a) minimizes the makespan. Constraints (7b) keep the relevant constraints of the original model into the master problem. Constraints (7c) introduce additional restrictions to ensure that the total area of the allocated parts in each batch cannot exceed the total area of the build chamber. Constraints (7d)–(7e) present the ranges of the decision variables.

5.2. The subproblem

By solving the master problem, we can obtain a solution that represents the allocation of parts to the different batches. Suppose $S = \{u_{ib}^*, z_b^*\}$ is the solution of the master problem, and $C_{\max}^*$ is the corresponding makespan of the master problem. We denote $\bar{I}_b = \{i \in I | u_{ib}^* = 1\}$ as the set of parts that are allocated to batch b . For each batch b , we now consider a corresponding subproblem to verify whether the parts can be feasibly packed into the build chamber. The subproblem is only used to test the feasibility of the solution S and the objective function has no impact on its feasibility, so we set the objective function as minimize 0. In addition, we denote the subproblem for batch b as sub(b), which can be described as follows.

$$[\text{sub}(b)] \quad \min \quad 0 \quad (8a)$$

$$\text{s.t.} \quad x_i + w_i \leq W \quad \forall i \in \bar{I}_b \quad (8b)$$

$$y_i + \ell_i \leq L \quad \forall i \in \bar{I}_b \quad (8c)$$

$$\text{left}_{ij} + \text{left}_{ji} + \text{below}_{ij} + \text{below}_{ji} \geq 1 \quad \forall i, j \in \bar{I}_b, i \neq j \quad (8d)$$

$$x_i + w_i \leq x_j + W(1 - \text{left}_{ij}) \quad \forall i, j \in \bar{I}_b, i \neq j \quad (8e)$$

$$y_i + \ell_i \leq y_j + L(1 - \text{below}_{ij}) \quad \forall i, j \in \bar{I}_b, i \neq j \quad (8f)$$

$$\text{left}_{ij}, \text{below}_{ij} \in \{0, 1\} \quad \forall i, j \in \bar{I}_b, i \neq j \quad (8g)$$

$$x_i, y_i \geq 0 \quad \forall i \in \bar{I}_b \quad (8h)$$

Constraints (8b)–(8c) ensure that each part in this batch cannot be placed outside the build chamber in either the horizontal or vertical direction. Constraints (8d)–(8f) guarantee that any two parts in this batch have at least one relative positional relationship without overlapping. Constraints (8g)–(8h) present the ranges of decision variables.

Obviously, if all of the subproblems are feasible, then the solution $S = \{u_{ib}^*, z_b^*\}$ of the master problem is optimal to the original problem, and thus we solve the SMSAM-2DP problem optimally. When there exists at least one subproblem that is infeasible, then we should exclude the allocation plan of parts for the corresponding batch from further consideration, that is, we generate combinatorial Benders cuts to forbid the above allocation plan of parts and add such cuts to the master problem.

5.3. The algorithmic outline of Algorithm CBD

The general algorithmic outline of Algorithm CBD is summarized as in Algorithm 5.1.

Algorithm 5.1 The algorithmic outline of Algorithm CBD

```

1: Initialization: flag ← 1.
2: while flag = 1 do
3:   FeasibleBatchcounter ← 0.
4:   Solve master problem (7) to obtain solution  $x^*$ , with corresponding number of batches  $B$ .
5:   if the MP is feasible then
6:     for  $b = 1$  to  $B$  do
7:       Solve corresponding subproblem (8) for batch  $b$ , i.e., sub(b).
8:       if sub(b) is infeasible then
9:         Add corresponding combinatorial Benders cuts to master problem.
10:        break
11:      else
12:        FeasibleBatchcounter ← FeasibleBatchcounter + 1.
13:    if FeasibleBatchcounter =  $B$  then
14:      All subproblems are feasible. Set flag ← 0.
15:      Output current solution  $x^*$ .
16:    else
17:      The original problem is infeasible.
18:    break

```

5.4. Combinatorial benders cuts

In this section, we propose various approaches to generate different types of combinatorial Benders cuts, including the no-good cuts, initial cuts and minimal infeasible subset-based cuts.

5.4.1. No-good cuts

Each time when the subproblem for batch b , i.e., sub(b), is infeasible, we know that the corresponding set of parts cannot be allocated to the same batch. Let $\tilde{I}_b = \{i \in I | u_{ib}^* = 1, \text{ and sub}(b) \text{ is infeasible}\}$, then we can add the following combinatorial Benders cut into the master problem:

$$\sum_{i \in \tilde{I}_b} u_{ib} \leq |\tilde{I}_b| - 1 \quad \forall b \in B \quad (9)$$

Using the above Benders cuts, i.e., Constraints (9), we can forbid the corresponding infeasible allocation plan by removing one part from the corresponding set of $\tilde{I}_b$.

To illustrate, as shown in Fig. 2, suppose that we have 6 parts, and the solution obtained by solving the master problem is $u_{11}^* = u_{21}^* = u_{31}^* = u_{51}^* = 1$ and $u_{42}^* = u_{62}^* = 1$; that is, parts 1, 2, 3, and 5 are allocated to batch 1, and parts 4 and 6 are allocated to batch 2. However, the corresponding sub(1) problem is infeasible, thus we can generate the following Benders cut:

$$u_{1b} + u_{2b} + u_{3b} + u_{5b} \leq 3 \quad \forall b \in B \quad (10)$$

The above Benders cuts are generated by the infeasible allocation plan obtained by solving the master problem, which are denoted as *no-good cut*. When the number of parts allocated into an infeasible batch is large, the corresponding Benders cut might be quite loose. As a result, if we simply implement such kind of Benders cut, a large amount of iterations may be needed to achieve an optimal solution of the original problem.

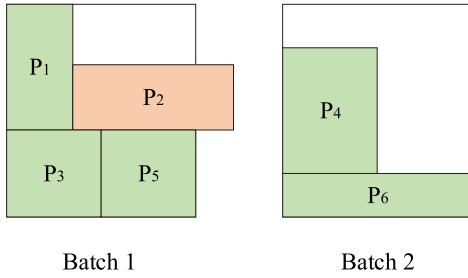


Fig. 2. No-good cut.

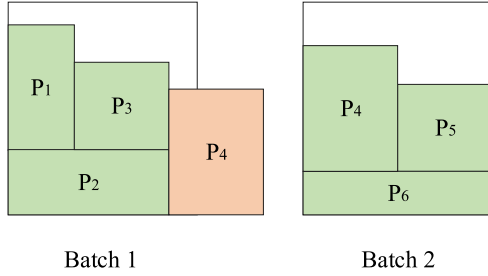


Fig. 3. Next-fit-based cut.

5.4.2. Initial cuts

We have added a new type of constraint to ensure that the total area of the allocated parts in each batch cannot exceed the total area of the build chamber, i.e., Constraints (7c), and we denote this type of cut as an *area-restriction cut*. With area-restriction cuts, we can avoid those trivial infeasible batches derived from the solution of the master problem.

In addition, in order to obtain other infeasible batches, we employ a *next-fit-based heuristic method* to identify the set of parts that should not be placed together in the same batch, and denote this type of cut as the *next-fit-based cut*. To be specific, according to any given order of parts, we pack each part subsequently into the current batch and solve the corresponding subproblem to check its feasibility. If the current batch is still feasible, we continue such process by adding the next unpacked part into this batch. Otherwise, we obtain an infeasible set of parts that cannot be placed together and generate a corresponding Benders cut. Then, we open a new batch to place this part, and repeat the above process to generate new Benders cuts.

To illustrate, suppose that we have a set of parts that have been ordered as $P_1, P_2, \dots, P_6$, and we pack these parts subsequently according to their indices. From Fig. 3, we can see that parts P_1, P_2 and P_3 can be feasibly packed into a batch. However, part P_4 cannot be packed into the same batch together with the three packed parts. Therefore, we can obtain an infeasible set of parts that cannot be packed together into the same batch, and the following Benders cut can be generated:

$$u_{1b} + u_{2b} + u_{3b} + u_{4b} \leq 3 \quad \forall b \in B \quad (11)$$

Then, we open a new batch and pack the remaining parts starting from P_4 , and continue the above process until all parts have been packed into batches. Since parts P_4, P_5 and P_6 can be placed together in batch 2, we do not have any other next-fit-based cut for the given example, and we stop this process.

With the above next-fit-based cuts, we can not only exclude some of the infeasible allocation plans from further consideration, but also provide an upper bound on the number of batches that need to be opened, as the final allocation plan obtained by this heuristic method is a feasible solution to the SMSAM-2DP problem.

5.4.3. Minimal infeasible subset cuts

In order to strengthen the generated no-good cuts, one natural idea is to examine all subsets of the parts in the corresponding infeasible batch, and check whether any of its subset is still infeasible. Inspired by the work of Verstichel et al. (2015), we employ the so-called *minimal infeasible subset cuts* (or MIS cuts for short) as in Algorithm 5.2. The main idea of generating the MIS cuts is as follows: **Given any infeasible batch with N parts, we start enumerating each subset of this batch with a cardinality of 2, i.e., each subset has only 2 parts, and check its feasibility by solving the corresponding subproblem.** Each time when an infeasible subset is obtained, we generate a new Benders cut with respect to this subset, and all supersets that include this subset will be excluded from further consideration. We continue such process by gradually increasing the cardinality of the subset from 2 to N until no more action can be made.

Algorithm 5.2 The algorithm for generating the MIS cuts

- 1: Given an infeasible batch $\tilde{I}_b$ with N parts.
- 2: Initialization: $n_s \leftarrow 2$; set of infeasible subsets $F \leftarrow \emptyset$.
- 3: **if** $n_s < N + 1$ **then**
- 4: Generate all subsets with a cardinality of n_s . Denote it as set $\tilde{I}_b^{n_s}$
- 5: **if** $\tilde{I}_b^{n_s} \neq \emptyset$ **then**
- 6: **for** Each subset $S \in \tilde{I}_b^{n_s}$ **do**
- 7: **if** S is a superset of some infeasible subset in F **then**
- 8: $\tilde{I}_b^{n_s} \leftarrow \tilde{I}_b^{n_s} \setminus S$.
- 9: **else**
- 10: Solve the subproblem for subset S .
- 11: **if** The corresponding subproblem is infeasible **then**
- 12: $F \leftarrow F \cup \{S\}$.
- 13: $\tilde{I}_b^{n_s} \leftarrow \tilde{I}_b^{n_s} \setminus S$.
- 14: **else**
- 15: $n_s \leftarrow n_s + 1$.

To illustrate, we consider an example with 6 parts as shown in Fig. 4. We first generate all subsets with $n_s = 2$, with a total number of $C_6^2 = 15$ combinations. Since all these subsets are feasible, we update $n_s = 3$ and continue checking the feasibility of all subsets from $C_6^3 = 20$ combinations, among which we find that there exist two subsets that are infeasible, i.e., $\{1, 2, 6\}, \{2, 4, 6\}$. Then, we update $n_s = 4$ and continue the above process except that all the supersets that include $\{1, 2, 6\}$ or $\{2, 4, 6\}$ as a subset will be excluded. For example, set $\{1, 2, 3, 6\}$ will not be checked since it contains $\{1, 2, 6\}$ as a subset. By implementing the above process until no more subset needs to be considered, we finally obtain all MIS cuts from the given infeasible batch.

Although the above approach can output all infeasible subsets, the computational time is expected to be exponentially increasing as we may need to enumerate all of its possible subsets, and thus it may not be practical to implement such process when the total number of parts in the given infeasible batch is large.

5.4.4. MIS-based heuristic cut

In order to generate effective MIS cuts and avoid the time-consuming characteristic of the above enumeration based process, we propose the so-called *MIS-based heuristic method*. **The main idea of such method is to generate part or all of the minimal infeasible subsets. To be specific, for any given infeasible batch with N parts, we consider the following three different types of MIS-based heuristic cuts: In the first one, we only find those infeasible subsets with $n_s = N - 1$, and call the generated cuts as the *one-layer MIS-based heuristic cut*. In the second one, we only find those infeasible sets with $n_s = N - 1$ and $n_s = N - 2$, and call the corresponding cuts as the *two-layer MIS-based heuristic cut*. Similarly, we call the MIS cuts proposed in Section 5.4.3 as the *all-layer MIS-based heuristic cut*.**



Fig. 4. An illustrative example for generating the MIS cuts.

According to our preliminary computational experiments, we find that the above MIS-based heuristic cut can be used to effectively reduce the search space and speed up the solution process. However, for large-sized instances, the computation time remains unsatisfactory. To accelerate the solution process of our proposed Algorithm CBD, we propose several acceleration strategies in the next section.

5.5. Acceleration strategies

In this section, we provide three different acceleration strategies as follows.

5.5.1. Strategy 1: Obtaining tighter bounds on the number of batches

As we know, the solution process of the master problem can be accelerated if we provide tighter bounds on the number of batches. Let $LB(I)$ be the lower bound on the number of batches for a given set of parts I . Obviously, we know that $LB(I) \geq \lceil \sum_{i \in I} w_i \ell_i / WL \rceil$, that is, the number of batches must be greater than the total area of parts divided by the area of the build chamber.

Meanwhile, we let $OPT(I)$ be the optimal number of bins for the corresponding two-dimensional bin packing problem to pack all of the parts into the build chamber, then $OPT(I)$ is also a lower bound on the number of batches, that is, $LB(I) \geq OPT(I)$. As we know, there has been a considerable literature on designing different approximation algorithms for the two-dimensional bin packing problem. Let $ALG(I)$ be the corresponding number of bins used in an approximation algorithm ALG. If α is the corresponding approximation ratio of ALG, then we know that $OPT(I) \geq \lceil ALG(I) / \alpha \rceil$. In this work, we use the so-called hybrid first fit (HFF) algorithm, which was proposed by Chung et al. (1982), to calculate a lower bound on the number of batches.

Combining the above two observations, we have the following result.

$$LB = \max \left\{ \left\lceil \frac{\sum_{i \in I} w_i \ell_i}{WL} \right\rceil, \left\lceil \frac{HFF(I)}{\alpha} \right\rceil \right\} \quad (12)$$

5.5.2. Strategy 2: Introducing a secondary objective

Through some experimentation, we find that the infeasibility of the subproblem is usually caused by allocating too many parts into the same batch, which may lead to unavoidable overlapping of parts. In order to prevent such situations, we introduce a secondary objective into the master problem so as to minimize the deviation of the number of parts across all batches while the primary objective remains minimizing the makespan. That is, we aim at distributing the parts into batches as equally as possible with the same makespan so that the number of parts in each batch will not be too large. The revised master problem is the following.

$$\min C_{\max} + \varepsilon \cdot (\bar{O} - O) \quad (13a)$$

$$s.t. \text{ Constraints (7b)–(7e)} \quad (13b)$$

$$O_b = \sum_{i \in I} u_{ib} \quad \forall b \in B \quad (13c)$$

$$\bar{O} \geq O_b \quad \forall b \in B \quad (13d)$$

$$O \leq O_b \quad \forall b \in B \quad (13e)$$

The coefficient $\varepsilon > 0$ in the secondary objective is a sufficiently small value so that the primary objective will not be affected by the secondary objective. Constraints (13c) define the number of parts in each batch b . Constraints (13d) and (13e) define the maximum and minimum numbers of parts across all batches.

5.5.3. Strategy 3: Applying steinberg's theorem

During the implementation of our proposed Algorithm CBD, we need to solve the subproblem for each batch to verify whether the solution obtained by the master problem is feasible for all batches. In fact, if the allocated parts in a given batch satisfy the conditions of Steinberg's Theorem as shown in Theorem 1 (Steinberg, 1997), then we can directly apply this theorem to verify whether the allocated parts can be feasibly packed into this batch, that is, we can simply calculate and compare the above values instead of solving the subproblem, and thus can speed up the solution process of Algorithm CBD.

Table 1

Parameters of the additive manufacturing machines.

Machine type	VT (h/cm ³)	HT (h/cm)	ST (h)	L (cm)	W (cm)	H (cm)
Small (S)	0.030864	0.7	2	15	15	32.5
Medium (M)	0.030864	0.7	2	17.5	17.5	32.5
Large (L)	0.030864	0.7	2	20	20	32.5

Table 2

Parameters for each combination.

Combination	1	2	3	4	5	6	7	8	9	10	11	12
Machine type	S	M	L	S	M	L	S	M	L	S	M	L
Number of parts	15	15	15	20	20	20	30	30	30	40	40	40

6. Computational experiments

To test the performance of the MILP model and our proposed Algorithm CBD, we conduct extensive computational experiments on various instances with part-related data from Che et al. (2021). To perform these experiments, we use Gurobi Optimizer 9.5.1 to solve the MILP model and the Python programming language to implement Algorithm CBD on a computer with a 2.8 GHz Intel Core i7 processor and 16 GB of RAM running on the Windows 10 operating system. In addition, for each instance, we set a time limit of 7200 s for both the MILP model and Algorithm CBD so as to better compare their performance within similar computational time.

6.1. Computational environments

In the work of Che et al. (2021), the authors have provided a dataset for parts with different orientations and various sizes. In this work, we choose the first orientation of each part in their dataset and output the corresponding characteristics of this part including its height, length, width and volume. We then randomly generate various parts based on the above part-related dataset, in which repeated selections are allowed. Meanwhile, we derive the additive machine-related parameters from the work of Kucukkoc (2019), including the forming time, powder layering time, and setup time of the additive machine. Note that the build chamber of additive machine in Che et al. (2021) and Kucukkoc (2019) usually have large capacities, and thus it may be the case that all parts can be packed into a single batch, which may make the scheduling decisions become unnecessary. In order to prevent such situations, by referring to the “dimensions.com” website (Powell, 2021), we consider three different types of additive machines with the parameters shown in Table 1.

To test the performance of our proposed method with a diversified dataset, we consider the following combinations of the number of parts n and the type of machines:

$$\{(n, \text{type}) : n \in \{15, 20, 30, 40\}, \text{type} \in \{S, M, L\}\}.$$

For each combination, we randomly generate 10 instances, for a total of $4 \times 3 \times 10 = 120$ instances. The detailed parameters of each combination are shown in Table 2.

6.2. Evaluation of components in Algorithm CBD

In order to identify the impact of each component in Algorithm CBD, we conduct the following series of computational experiments.

6.2.1. Comparison of performance with different acceleration strategies

As mentioned in Section 5.5, we have introduced three different acceleration strategies to speed up the solution procedure of the master problem. i.e., obtaining tighter bounds (strategy 1), introducing a secondary objective (strategy 2), and applying Steinberg's Theorem (strategy 3). In this section, we compare the efficiency of these three

strategies based on instances with small machine type and the numbers of parts being equal to 15 and 20, i.e., combinations 1 and 4. The results are depicted in Tables 3 and 4. In these tables, we use “MP Iter” and “SP Iter” to denote the numbers of iterations for the master problem and subproblems, respectively. The value “x” with a form of “(x/10)” in the last row of Tables 3 and 4 represents the number of instances in which the result is better than the results without the acceleration strategies. In addition, if an instance cannot be solved optimally within the time limit, we record the corresponding optimality gap (denoted by ‘Gap’), and use 7200 s as its computation time to compute the corresponding average computational time (denoted by ‘Avg’). Then, we add a ‘>’ sign before the calculated value.

From Tables 3 and 4, we can see that these strategies and their combinations can significantly reduce the CPU time and the number of iterations, which indicates the effectiveness of our proposed acceleration strategies for the SMSAM-2DP problem. To be specific, by implementing strategy 1, we can observe a better improvement in the computation time, and also a reduction in the number of iterations for some instances; By implementing strategy 2, we can see that the computation time and the number of iterations are both reduced as the number of parts in each batch become more balanced; By implementing strategy 3, the number of iterations for subproblems can be reduced as it can be more efficient to determine whether a subproblem is feasible by applying Steinberg's Theorem. Finally, by combining all acceleration strategies, we can obtain the best performance, and its average CPU time is only about half of the time without implementing any acceleration strategy. In what follows, we only consider the performance of Algorithm CBD which includes all of these acceleration strategies.

6.2.2. Comparison of performance with different types of benders cuts

In order to test the efficiency of the different types of Benders cuts that we have proposed in Section 5.4, we use the same instances as above, i.e., combinations 1 and 4, and compare the performance of Algorithm CBD with different types of Benders cuts. We denote Algorithm CBD0 as the combinatorial Benders decomposition algorithm that only uses the no-good cuts, and Algorithm CBD1 as the one that uses both the no-good cuts and the initial cuts. In addition to including the above two types of Benders cuts, we also consider different combinatorial Benders decomposition algorithms with one-layer, two-layer and all-layer MIS cuts, and refer to these as Algorithms CBD2, CBD3 and CBD4, respectively. Tables 5 and 6 show the corresponding results. From these two tables, we can see that any of these algorithms can perform significantly better than solving the MILP model directly by Gurobi.

Meanwhile, we can see that the number of iterations for the subproblems by Algorithm CBD1 is increased compared to the one by Algorithm CBD0, while the computation time of Algorithm CBD1 is generally smaller than Algorithm CBD0. The reason may be the following: by imposing the initial cuts, the number of infeasible solutions to the master problem can be reduced, and tighter upper bounds on the number of batches can be obtained.

Moreover, for the instances in combination 1 with $n = 15$, we can see that by incorporating the MIS-based heuristic cuts, the number of iterations for the master problem in Algorithms CBD2, CBD3 and CBD4 can be reduced compared to the number of iterations in Algorithms CBD0 and CBD1, while the difference of computational time between these algorithms is not significant. When the number of parts increases, say $n = 20$, we can see that both the number of iterations for the master problem and the computational times of Algorithms CBD2, CBD3 and CBD4 go down significantly. These results indicate that the no-good cuts become quite loose when the problem size increases, and the MIS-based heuristic cuts could be quite effective in solving the SMSAM-2DP problem. Based on the above results, in what follows we only consider the performance of Algorithm CBD that includes different types of MIS-based heuristic cuts, i.e., Algorithms CBD2, CBD3 and CBD4.

Table 3
Comparison of performance between different acceleration strategies for combination 1.

Instance	MILP			Opt	Algorithm CBD without any strategy			Algorithm CBD with strategy 1			Algorithm CBD with strategy 2			Algorithm CBD with strategy 3			Algorithm CBD with all strategies		
	Obj	Time	Gap ^o		MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time
1	72.77	960.4	0.00%	72.77	1	17	0.3	1	17	0.3	1	17	0.5	1	11	0.4	1	11	0.3
2	63.23	1127.2	0.00%	63.23	7	100	6.0	5	79	3.3	3	47	2.1	7	91	8.5	4	56	1.8
3	40.72	5767.6	0.00%	40.72	2	28	3.9	4	52	4.6	2	25	2.0	2	18	3.5	2	19	1.8
4	74.28	1453.2	0.00%	74.28	15	223	17.5	17	249	19.8	17	255	23.3	15	203	17.2	17	227	18.4
5	77.52	107.5	0.00%	77.52	3	31	1.8	3	31	1.5	3	31	1.8	3	21	1.9	3	21	1.1
6	77.57	203.0	0.00%	77.57	10	133	9.2	6	83	5.6	12	152	9.5	10	120	8.6	8	98	5.4
7	67.56	–	1.35%	67.56	22	289	71.9	21	287	60.8	24	314	72.6	22	263	65.8	22	269	50.2
8	73.12	89.9	0.00%	73.12	44	582	24.6	50	648	24.3	41	523	23.0	44	572	23.8	35	430	17.5
9	75.89	710.9	0.00%	75.89	10	137	13.5	11	135	12.3	10	129	12.4	10	124	10.8	9	104	7.5
10	48.75	1126.2	0.00%	48.75	37	466	23.0	30	424	17.8	32	405	19.6	37	454	21.9	33	377	15.0
Avg	67.14	>1874.6	0.14%	67.14	15.1	200.6	17.2	14.8 (4/10)	200.5 (5/10)	15.1 (8/10)	15 (3/10)	189.8 (5/10)	16.7 (5/10)	15.1 (0/10)	187.7 (10/10)	16.2 (7/10)	13.4 (5/10)	161.2 (9/10)	11.9 (9/10)

–: for these instances, the optimal solutions cannot be obtained within 7200 s.

Table 4
Comparison of performance between different acceleration strategies for combination 4.

Instance	MILP			Opt	Algorithm CBD without any strategy			Algorithm CBD with strategy 1			Algorithm CBD with strategy 2			Algorithm CBD with strategy 3			Algorithm CBD with all strategies		
	Obj	Time	Gap ^o		MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time
1	97.45	–	6.00%	97.45	121	1785	255.8	151	2129	172.2	102	1492	193.9	121	1742	234.4	112	1580	121.9
2	93.03	–	5.45%	93.03	16	256	29.5	34	520	66.8	17	270	17.2	16	239	23.2	5	100	4.5
3	92.45	–	5.03%	92.45	58	827	115.9	70	1024	86.2	74	1049	148.4	58	763	132.1	70	944	86.1
4	79.59	–	4.62%	79.59	31	538	82.6	28	512	50.3	35	508	69.9	31	504	89.8	34	451	46.0
5	81.92	–	6.20%	81.64	48	828	75.5	24	430	31.1	25	442	56.0	48	776	81.1	26	415	29.1
6	92.15	–	11.60%	87.42	3	42	4.5	3	42	3.1	3	42	4.0	3	34	4.9	3	34	3.4
7	83.51	–	4.81%	83.51	12	202	831.6	3	54	29.7	6	111	60.0	12	193	850.6	5	62	11.5
8	78.95	–	7.22%	78.95	427	4421	518.1	422	4314	423.0	425	4344	561.5	427	4413	521.5	434	4469	473.0
9	90.94	–	3.12%	90.94	21	305	116.9	23	330	26.9	21	303	56.6	21	283	128.3	21	273	26.4
10	89.64	–	2.93%	89.36	9	164	23.4	16	211	16.8	9	133	20.2	9	147	25.0	8	103	11.1
Avg	87.96	> 7200	5.70%	87.43	74.6	936.8	205.4	77.4 (4/10)	956.6 (4/10)	90.6 (9/10)	71.7 (4/10)	869.4 (7/10)	118.8 (8/10)	74.6 (0/10)	909.4 (10/10)	209.1 (2/10)	71.8 (5/10)	843.1 (8/10)	81.3 (10/10)

–: for these instances, the optimal solutions cannot be obtained within 7200 s.

Table 5
Comparison of performance between different types of Benders cut for combination 1.

Instance	MILP			Opt	Algorithm CBD0			Algorithm CBD1			Algorithm CBD2			Algorithm CBD3			Algorithm CBD4		
	Obj	Time	Gap ^o		MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time
1	72.77	960.4	0.00%	72.77	1	2	0.1	1	11	0.2	1	11	0.3	1	11	0.2	1	11	0.2
2	63.23	1127.2	0.00%	63.23	9	20	1.9	8	28	1.6	4	56	1.8	3	88	2.0	2	73	1.0
3	40.72	5767.6	0.00%	40.72	1	3	0.1	2	11	1.3	2	19	1.8	2	47	5.0	2	133	2.5
4	74.28	1453.2	0.00%	74.28	49	98	37.7	49	107	29.8	17	227	18.4	12	474	14.5	11	804	18.4
5	77.52	107.5	0.00%	77.52	7	21	2.9	5	21	1.7	3	21	1.1	3	25	1.1	3	23	1.1
6	77.57	203.0	0.00%	77.57	28	61	15.1	30	77	12.3	8	98	5.4	6	212	6.7	11	623	15.3
7	67.56	–	1.35%	67.56	41	90	61.9	43	104	47.2	22	269	50.2	20	889	64.9	20	3335	104.0
8	73.12	89.9	0.00%	73.12	129	387	87.8	123	379	46.9	35	430	17.5	26	635	17.0	25	777	21.4
9	75.89	710.9	0.00%	75.89	19	50	12.9	19	59	8.3	9	104	7.5	9	282	11.1	9	773	20.3
10	48.75	1126.2	0.00%	48.75	89	266	62.5	79	244	31.1	33	377	15.0	27	762	18.8	27	845	22.4
Avg	67.14	>1874.6	0.14%	67.14	37.3	99.8	28.3	35.9	104.1	18.1	13.4	161.2	11.9	10.9	342.5	14.1	11.1	739.7	20.7

–: for these instances, the optimal solutions cannot be obtained within 7200 s.

Table 6
Comparison of performance between different types of Benders cut for combination 4.

Instance	MILP			Opt	Algorithm CBD0			Algorithm CBD1			Algorithm CBD2			Algorithm CBD3			Algorithm CBD4		
	Obj	Time	Gap ^o		MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time	MP Iter	SP Iter	Time
1	97.45	–	6.00%	97.45	123	482	313.2	134	532	287.5	112	1580	121.9	106	4273	92.8	124	10866	144.6
2	93.03	–	5.45%	93.03	21	75	8.7	21	84	10.3	5	100	4.5	10	386	13.2	8	738	11.0
3	92.45	–	5.03%	92.45	230	713	727.5	227	719	572.4	70	944	86.1	57	2039	96.5	61	3714	58.6
4	79.59	–	4.62%	79.59	96	356	90.1	104	395	87.8	34	451	46.0	22	846	42.4	21	1783	49.7
5	81.92	–	6.20%	81.64	44	140	23.5	45	157	22.2	26	415	29.1	23	982	41.0	34	2814	34.9
6	92.15	–	11.60%	87.42	3	9	1.2	3	20	1.4	3	34	3.4	3	77	4.7	3	285	7.6
7	83.51	–	4.81%	83.51	5	14	2.5	6	29	2.7	5	62	11.5	9	383	51.8	15	4287	111.9
8	78.95	–	7.23%	78.95	589	1767	812.4	588	1775	667.8	434	4469	473.0	424	12729	673.2	428	23038	410.8
9	90.94	–	3.12%	90.94	118	450	197.5	128	501	188.7	21	273	26.4	16	435	31.3	12	648	20.8
10	89.64	–	2.93%	89.36	41	144	39.2	47	168	27.8	8	103	11.1	7	182	15.1	7	646	19.8
Avg	87.96	> 7200	5.70%	87.43	127.0	415.0	221.6	130.3	438.0	186.9	71.8	843.1	81.3	67.7	2233.2	106.2	71.3	4881.9	87.0

–: for these instances, the optimal solutions cannot be obtained within 7200 s.

Table 7

Comparison of performance between Algorithms CBD2, CBD3, CBD4 and the MILP model with $n = 15$.

Combination	Instance	MILP				Algorithm CBD2		Algorithm CBD3		Algorithm CBD4	
		Optimality	$C_{\max}$	Time	Gap ^o	$C_{\max}$	Time	$C_{\max}$	Time	$C_{\max}$	Time
1	1	Optimal	72.77	960.4	0.00%	72.77	0.3	72.77	0.2	72.77	0.2
	2	Optimal	63.23	1127.2	0.00%	63.23	1.8	63.23	2.0	63.23	1.0
	3	Optimal	74.28	1453.2	0.00%	74.28	18.4	74.28	14.5	74.28	18.4
	4	Optimal	77.52	107.5	0.00%	77.52	1.1	77.52	1.1	77.52	1.4
	5	Optimal	73.12	89.9	0.00%	73.12	17.5	73.12	17.0	73.12	21.4
	6	Optimal	75.89	710.9	0.00%	75.89	7.5	75.89	11.1	75.89	20.3
	7	Optimal	48.75	1126.2	0.00%	48.75	15.0	48.75	18.8	48.75	22.4
	8	Optimal	98.88	411.7	0.00%	98.88	13.3	98.88	5.1	98.88	4.1
	9	Optimal	89.59	111.2	0.00%	89.59	10.8	89.59	5.1	89.59	3.3
	10	Optimal	96.80	1412.3	0.00%	96.80	11.5	96.80	4.3	96.80	3.7
2	1	Optimal	68.41	2882.0	0.0%	68.41	19.4	68.41	18.3	68.41	29.7
	2	Optimal	59.06	727.9	0.00%	59.06	10.0	59.06	14.1	59.06	18.9
	3	Optimal	69.76	614.2	0.00%	69.76	0.5	69.76	0.5	69.76	0.4
	4	Feasible	73.70	–	1.42%	73.70	79.7	73.70	70.8	73.70	106.3
	5	Feasible	67.20	–	1.04%	67.20	52.2	67.20	40.2	67.20	84.4
	6	Optimal	71.16	3960.3	0.00%	71.16	111.7	71.16	67.5	71.16	102.6
	7	Feasible	45.28	–	3.71%	45.28	421.1	45.28	569.4	45.28	607.2
	8	Optimal	79.94	346.4	0.00%	79.94	13.8	79.94	9.4	79.94	8.6
	9	Optimal	75.79	53.9	0.00%	75.79	10.0	75.79	3.4	75.79	2.2
	10	Optimal	72.56	113.6	0.00%	72.56	4.9	72.56	2.6	72.56	2.1
3	1	Feasible	67.62	–	0.91%	67.62	6.1	67.62	3.9	67.62	3.9
	2	Optimal	58.43	306.9	0.00%	58.43	2.9	58.43	4.7	58.43	3.0
	3	Optimal	69.26	3362.9	0.00%	69.26	2.2	69.26	5.4	69.26	3.2
	4	Feasible	71.60	–	0.12%	71.60	0.5	71.60	0.8	71.60	1.0
	5	Optimal	65.01	79.5	0.00%	65.01	0.4	65.01	0.6	65.01	0.7
	6	Optimal	70.87	44.4	0.00%	70.87	2.4	70.87	49.8	70.87	53.4
	7	Optimal	43.10	233.4	0.00%	43.10	25.0	43.10	29.4	43.10	24.4
	8	Optimal	72.34	165.2	0.00%	72.34	6.1	72.34	4.5	72.34	4.8
	9	Optimal	70.99	10.6	0.00%	70.99	2.3	70.99	1.4	70.99	5.0
	10	Optimal	72.56	83.7	0.00%	72.56	4.0	72.56	3.3	72.56	1.3
Avg		/	70.52	>1883.2	0.24%	70.52	29.1	70.52	32.6	70.52	38.6

–: for these instances, the optimal solutions cannot be obtained within 7200 s.

6.2.3. Comparison of performance between Algorithms CBD2, CBD3, CBD4 and the MILP model

To further test the performance of our proposed MILP model and combinatorial Benders decomposition algorithms, we conduct a series of experiments on more general combinations of instances with different sizes.

We first consider those small-sized instances with $n = 15$ and $n = 20$, i.e., combinations 1 to 6, and Tables 7–8 show the corresponding results. From these two tables, we can see that all the instances in these 6 combinations can be solved optimally by Algorithms CBD2, CBD3 and CBD4, and most instances can also be solved optimally by the MILP model. It should be noted that the average CPU running times of the different combinatorial Benders decomposition algorithms are significantly smaller than the one obtained by solving the MILP model with Gurobi. In addition, we found that the average CPU running time of Algorithm CBD4 is longer than the times of Algorithms CBD2 and CBD3 when the number of parts is $n = 15$. When $n = 20$, we can see that the average CPU running time of Algorithm CBD2 becomes longer than the running times of the other two algorithms. The reason for such phenomenon may be that when the number of parts is small, the optimal solution can be found with a considerably smaller number of iterations for the master problem. As we know, Algorithms CBD4 needs to enumerate all possible infeasible subsets to obtain the associated Benders cuts, and thus the computation time of Algorithm CBD4 is longer than the times of Algorithms CBD2 and CBD3. When the number of parts increases, the computation time for solving the master problem becomes larger for each iteration. Since Algorithm CBD2 only needs to find the one-layer subsets for each infeasible batch, which may be still quite loose, and thus more iterations of the master problem are required

and the computation time of Algorithm CBD2 becomes larger than the ones of Algorithm CBD3 and CBD4.

Then, we consider those large-sized instances with $n = 30$ and $n = 40$, i.e., combinations 7 to 12. According to our preliminary experiments, we find that the main computation time is spent on solving the master problem during each iteration, especially when the sizes of the instances increase. In fact, it may be unable to obtain the optimal solution of the master problem within a given time limit of 7200 s. Inspired by the work of Zohali et al. (2022), we introduce two additional parameters during the solution process of the master problem for these instances: (i) the maximum computation time T (in seconds) for solving the master problem; and (ii) the allowable optimality gap G of the master problem. Given these two parameters, we solve the master problem using an approximate approach instead of solving them optimally during each iteration so as to accelerate the solution process of the master problem. Note that the master problem is a relaxation of the original problem, if the solution of the master problem obtained by the above approximate approach is feasible to the original problem, then the real optimality gap of this solution for the original problem must be less than or equal to the corresponding optimality gap G for the master problem.

In particular, we design an effective method to dynamically adjust the values of parameters T and G as follows. Within the given time limit, we solve the master problem with initial parameters T and G , which then leads to one of the following four states: (i) Optimal, which means that an optimal solution of the master problem is found; this solution can then be passed to the subproblems, and we update $G = G - 0.01$ in the next iteration of the master problem; (ii) Feasible, which indicates that a feasible solution is found within the given time limit but the corresponding optimality gap exceeds the value of parameter

Table 8Comparison of performance between Algorithms CBD2, CBD3, CBD4 and the MILP model with $n = 20$.

Combination	Instance	MILP				Algorithm CBD2		Algorithm CBD3		Algorithm CBD4	
		Optimality	$C_{\max}$	Time	Gap ^a	$C_{\max}$	Time	$C_{\max}$	Time	$C_{\max}$	Time
4	1	Feasible	79.59	–	4.62%	79.59	82.6	79.59	55.8	79.59	113.9
	2	Feasible	81.92	–	6.20%	81.64	75.5	81.64	64.2	81.64	84.5
	3	Optimal	111.73	1724.2	0.00%	111.73	88.8	111.73	26.8	111.73	17.1
	4	Optimal	95.09	2276.4	0.00%	95.09	146.7	95.09	32.5	95.09	41.9
	5	Feasible	84.34	–	2.62%	84.34	111.4	84.34	59.8	84.34	55.4
	6	Optimal	124.33	547.0	0.00%	124.33	242.2	124.33	13.2	124.33	9.9
	7	Feasible	149.94	–	3.60%	149.94	34.9	149.94	18.3	149.94	13.7
	8	Feasible	112.46	–	8.15%	112.46	268.7	112.46	54.3	112.46	15.4
	9	Optimal	116.22	4750.1	0.00%	116.22	25.9	116.22	18.7	116.22	17.7
	10	Feasible	132.14	–	9.88%	132.14	24.7	132.14	31.7	132.14	30.9
5	1	Feasible	74.59	–	5.61%	74.31	29.2	74.31	31.1	74.31	42.8
	2	Feasible	77.11	–	6.03%	77.11	1180.6	77.11	930.3	77.11	1180.1
	3	Optimal	100.03	408.1	0.00%	100.03	48.5	100.03	18.5	100.03	13.6
	4	Feasible	85.85	–	3.96%	85.85	141.9	85.85	65.1	85.85	58.3
	5	Optimal	78.84	6779.5	0.00%	78.84	1731.2	78.84	70.1	78.84	26.7
	6	Optimal	110.29	319.1	0.00%	110.29	50.7	110.29	16.2	110.29	26.1
	7	Optimal	121.39	384.6	0.00%	121.39	116.8	121.39	40.9	121.39	8.7
	8	Feasible	92.22	–	4.45%	92.22	52.2	92.22	22.4	92.22	19.5
	9	Optimal	95.29	658.3	0.00%	95.28	98.6	95.28	27.9	95.28	30.4
	10	Optimal	111.20	2597.7	0.00%	111.20	177.8	111.20	18.1	111.20	14.6
6	1	Optimal	70.41	195.5	0.00%	70.41	537.6	70.41	514.4	70.41	706.0
	2	Feasible	72.46	–	0.70%	72.46	2.8	72.46	57.3	72.46	70.1
	3	Optimal	91.30	2190.7	0.00%	91.03	14.1	91.03	5.9	91.03	20.1
	4	Optimal	75.13	60.3	0.00%	75.13	168.4	75.13	94.3	75.13	51.7
	5	Optimal	74.53	36.0	0.00%	74.53	1137.7	74.53	91.6	74.53	11.4
	6	Optimal	103.39	60.2	0.00%	103.39	35.8	103.39	12.2	103.39	2.6
	7	Optimal	107.74	194.4	0.00%	107.74	13.1	107.74	4.6	107.74	3.3
	8	Optimal	87.87	3003.2	0.00%	87.87	72.2	87.87	18.0	87.87	12.9
	9	Optimal	87.68	92.3	0.00%	87.68	240.4	87.68	37.7	87.68	20.2
	10	Optimal	101.50	2600.8	0.00%	101.50	344.8	101.50	49.9	101.50	6.9
Avg		/	96.88	>3602.6	1.86%	96.86	243.2	96.86	83.4	96.86	90.9

–: for these instances, the optimal solutions cannot be obtained within 7200 s.

G . Then, we choose the best feasible solution and pass it on to the subproblems, and update $G = G + 0.01$ in the next iteration of the master problem; (iii) Unknown, which means that the solver cannot verify whether the master problem is feasible within the time limit. Then, we ignore the time limit and continue solving the master problem until a feasible solution is found. After that, we pass the found feasible solution to the subproblems and update $G = G + 0.01$ during the next iteration of the master problem; (iv) Infeasible, which indicates that the master problem is infeasible. Then, we stop the algorithm and claim that the original problem is also infeasible. Fig. 5 shows the detailed schematic of adjusting the parameters dynamically during the solution process of the master problem.

Through some experimentation, we find that Algorithm CBD4 achieves the best performance for solving these large-sized instances. Therefore, we perform a series of experiments to quantify the impact of different parameter settings on the performance of Algorithm CBD4. To be specific, we consider the following six values for each parameter:

$T \in \{3, 5, 8, 10, 12, 15\}$, and $G \in \{0, 0.02, 0.04, 0.06, 0.08, 0.1\}$,

and randomly select 15 large-sized instances. First, we conduct experiments with different values of parameter T and identify the optimal performing parameter T . Based on this, we proceed to experiment with various values of parameter G . Figs. 6 and 7 show the corresponding performance of Algorithm CBD4 under different parameters, from which we can find that when we set the initial values of parameters T and G to 8 s and 6% respectively, Algorithm CBD4 achieves comparatively better performance. Therefore, we set $T = 8$ and $G = 0.06$ in the remaining experiments.

Tables 9 and 10 show the computational results for the instances with $n = 30$ and $n = 40$, respectively. From these tables, we can see

that the average CPU running times of the MILP model are rather long, and the average gap for these instances is quite large, e.g., greater than 20% for the instances with $n = 40$. In particular, for some of the instances, e.g., the instances in combinations 10 and 11, the Gurobi solver cannot even find any feasible solution within the given time limit. Meanwhile, it seems that Algorithm CBD4 has the best performance on solving those large-sized instances, as it can obtain optimal solutions for most of the instances with $n = 30$, and some of the instances with $n = 40$. Even for those instances that cannot be solved optimally by Algorithm CBD4, the corresponding optimality gap for each instance is rather small compared to the other approaches, which indicates that the all-layer MIS-based heuristic cuts could effectively reduce the computation time of the master problem when the size of instances increases. In particular, the parameters of T and G are dynamically adjusted depending on the solving state of the master problem during each iteration. Therefore, in certain situations, it is possible for Algorithms CBD2, CBD3 and CBD4 to yield the same makespan while exhibiting different optimality gaps. For example, for the 8th instance of combination 8, Algorithms CBD2, CBD3 and CBD4 obtain the same makespan (i.e., 149.88). However, their respective optimality gaps are 4.71%, 4.62% and 2.95%, which are quite different. Obviously, the real optimality gap for the corresponding solution is at most 2.95%.

6.2.4. Comparison of performance between Algorithms GreedyPack, CBD and two SA-based algorithms

To evaluate the performance of our proposed Algorithm CBD more comprehensively, we also conduct a comparative analysis involving Algorithm GreedyPack and two simulated annealing based algorithms integrated with first-fit and best-fit skyline packing patterns, denoted

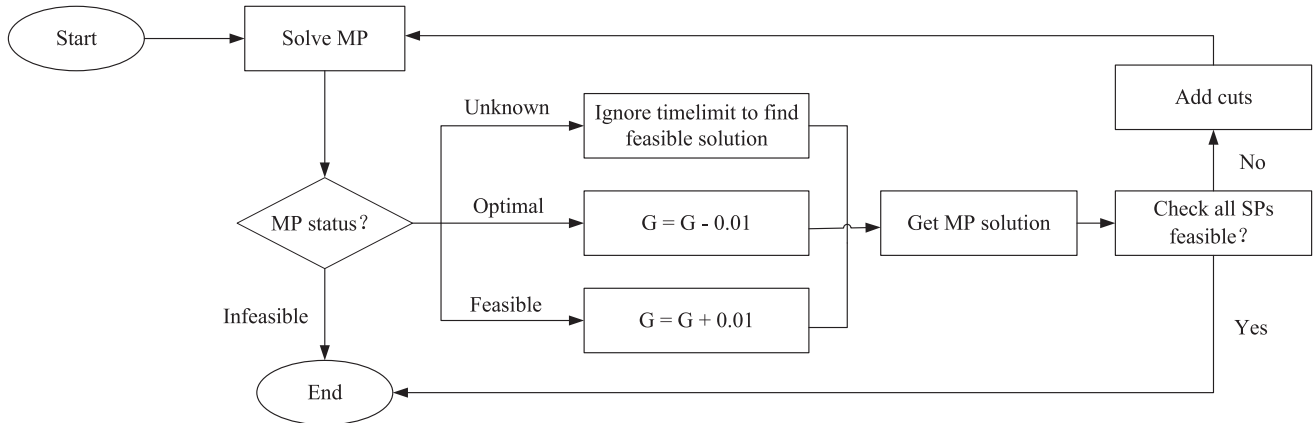
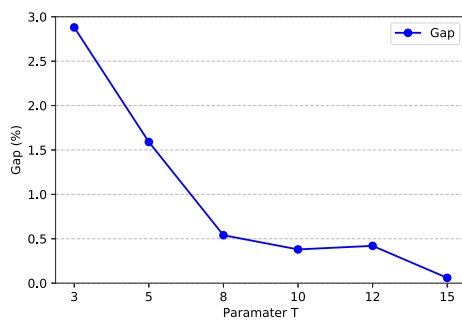
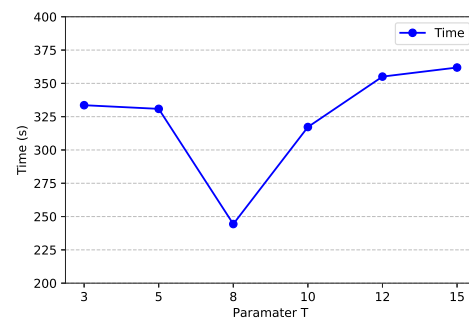


Fig. 5. Schematic of adjusting parameters dynamically during the solution process of MP.

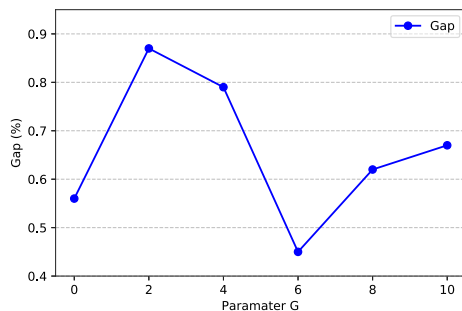


(a) Average gap versus parameter T

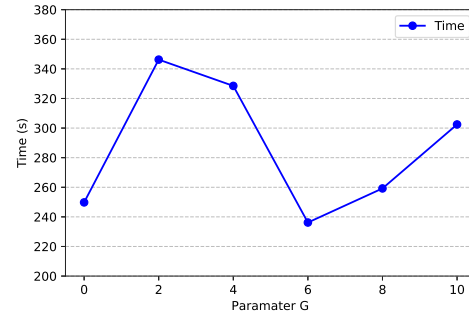


(b) Average time versus parameter T

Fig. 6. Performance of Algorithm CBD4 under different values of T .



(a) Average gap versus parameter G



(b) Average time versus parameter G

Fig. 7. Performance of Algorithm CBD4 under different values of G .

as Algorithm SA_FF and Algorithm SA_BF respectively, which were proposed by Che et al. (2021). For more details on the specific implementation of Algorithms SA_FF and SA_BF, we refer the reader to the work of Che et al. (2021). Based on our previous computational results, we know that Algorithms CBD2, CBD3 and CBD4 can optimally solve all instances of combinations 1-6 within the given time limit, and have the same optimal value for each instance. Therefore, we calculate the average values of their computational results for each combination and compare them with those obtained by Algorithms GreedyPack, SA_FF and SA_BF. Table 11 shows the corresponding computational results, in which their respective optimality gaps are obtained based on the objective values obtained by Algorithms GreedyPack, SA_FF, SA_BF and the optimal value.

Meanwhile, for combinations 7-12, there exist several instances that cannot be solved by specific combinatorial Benders decomposition-based algorithm within the given time limit. For example, the 8th instance of combination 9 cannot be solved by both Algorithms CBD2 and CBD3 within 7200 s, while Algorithm CBD4 can obtain a feasible solution with an optimality gap of 1.00%. Therefore, for the instances in combinations 7-12, we output the corresponding computational result for each instance obtained by Algorithms GreedyPack, SA_FF and SA_BF. In particular, as we have mentioned before, the solutions obtained by Algorithms CBD2, CBD3 and CBD4 may not be optimal as additional parameters (i.e., T and G) have been introduced in these experiments, so we first compare the objective values obtained by the MILP model, Algorithms CBD2, CBD3 and CBD4, and output the one with minimum value. Then we calculate the corresponding

Table 9

Comparison of performance between Algorithms GreedyPack, CBD, two SA-based algorithms and the MILP model with $n = 30$.

Comb	Ins	MILP			Algorithm CBD2			Algorithm CBD3			Algorithm CBD4			Algorithm GreedyPack			Algorithm SA_FF			Algorithm SA_BF			
		Opt	C _{max}	Time	Gap ^o	C _{max}	Time	Gap ^o	C _{max}	Time	Gap ^o	C _{max}	Time	Gap ^o	C _{max}	Time	Gap ^f	C _{max}	Time	Gap ^f	C _{max}	Time	Gap ^f
7	1	Feas	173.90	–	19.19%	173.90	1118.3	0.00%	173.90	261.0	0.00%	173.90	179.4	0.00%	208.30	<0.1	19.78%	190.76	0.1	9.69%	190.76	0.4	9.69%
	2	Feas	141.44	–	9.65%	141.44	916.7	9.81%	141.44	267.2	0.00%	141.44	268.2	0.00%	180.64	<0.1	27.72%	153.21	0.2	8.33%	150.51	0.8	6.42%
	3	Feas	156.63	–	11.15%	156.63	664.0	9.10%	156.63	290.8	2.78%	156.63	97.8	0.00%	192.67	<0.1	23.01%	172.36	0.1	10.04%	172.15	0.6	9.91%
	4	Feas	145.92	–	17.52%	142.80	1771.6	0.00%	142.80	689.7	0.00%	142.80	369.7	0.00%	177.55	<0.1	24.33%	161.13	0.1	12.83%	161.13	0.7	12.83%
	5	Feas	169.50	–	14.42%	169.50	543.2	0.79%	169.50	101.4	0.00%	169.50	59.5	0.00%	207.05	<0.1	22.15%	195.59	0.1	15.40%	195.59	0.7	15.40%
	6	Feas	162.97	–	15.74%	162.97	1421.5	1.77%	162.97	125.6	0.00%	162.97	42.6	0.00%	201.15	<0.1	23.43%	173.27	0.3	6.32%	176.67	0.8	8.41%
	7	Feas	147.29	–	8.78%	147.29	1832.5	0.00%	147.29	520.8	0.00%	147.29	297.1	0.00%	183.30	<0.1	24.45%	164.81	0.2	11.90%	165.02	1.1	12.04%
	8	Feas	184.02	–	18.19%	184.02	747.0	1.95%	184.02	136.9	0.00%	184.02	60.1	0.00%	215.09	<0.1	16.88%	197.13	0.2	7.12%	203.78	0.6	10.74%
	9	Feas	177.53	–	8.66%	176.83	2024.8	1.53%	176.83	595.3	2.00%	176.83	416.4	1.92%	218.23	<0.1	23.41%	210.63	0.1	19.11%	203.03	1.1	14.82%
	10	Feas	131.05	–	31.63%	125.90	1115.4	12.04%	125.90	1079.1	12.04%	125.90	843.7	12.13%	157.89	<0.1	25.41%	147.24	0.1	16.95%	147.24	0.3	16.95%
8	1	Feas	153.56	–	11.39%	148.76	2995.2	12.53%	148.76	667.7	9.81%	148.76	1164.8	7.70%	180.46	<0.1	21.31%	169.00	0.2	13.61%	169.00	0.9	13.61%
	2	Feas	129.14	–	9.06%	128.44	2057.0	5.85%	128.44	742.8	4.42%	128.44	252.6	0.00%	157.69	<0.1	22.77%	142.04	0.3	10.58%	139.79	1.8	8.84%
	3	Feas	143.53	–	3.83%	143.53	2248.7	7.62%	143.53	447.9	3.00%	143.53	54.1	0.00%	172.18	<0.1	19.96%	160.97	0.2	12.15%	160.73	1.8	11.98%
	4	Feas	123.16	–	2.76%	123.16	1362.0	7.69%	123.16	318.7	5.00%	123.16	99.0	0.00%	154.16	<0.1	25.17%	145.68	0.2	18.29%	148.20	1.3	20.34%
	5	Feas	144.60	–	3.97%	144.00	1081.7	5.93%	144.00	424.7	6.02%	144.00	120.6	1.89%	174.79	<0.1	21.38%	157.35	0.2	9.27%	159.34	2.7	10.65%
	6	Feas	147.07	–	9.53%	142.55	2880.6	4.66%	142.55	1211.6	2.41%	142.55	193.1	0.00%	177.47	<0.1	24.50%	160.17	0.2	12.36%	157.79	1.6	10.69%
	7	Opti	127.89	6687.1	0.00%	127.89	2666.0	5.91%	127.89	354.7	4.61%	127.89	50.0	0.00%	153.32	<0.1	19.88%	136.79	0.2	6.96%	137.49	1.9	7.51%
	8	Feas	151.17	–	7.21%	149.88	1229.3	4.71%	149.88	749.4	4.62%	149.88	714.4	2.95%	182.70	<0.1	21.90%	171.70	0.4	14.56%	171.70	1.4	14.56%
	9	Feas	159.99	–	9.95%	–	–	–	–	–	–	159.99	5315.0	11.27%	186.68	<0.1	16.68%	169.31	0.2	5.83%	170.01	1.8	6.26%
	10	Feas	109.89	–	19.61%	106.84	1440.6	12.87%	106.84	666.1	10.76%	106.84	579.7	10.45%	138.44	<0.1	29.58%	121.07	0.2	13.31%	121.07	0.7	13.31%
9	1	Feas	138.60	–	6.09%	–	–	–	–	–	–	–	–	–	166.56	<0.1	20.17%	151.70	0.2	9.45%	151.70	1.7	9.45%
	2	Feas	120.84	–	2.81%	–	–	–	120.84	588.2	0.69%	120.84	141.1	0.00%	146.84	<0.1	21.51%	131.39	0.4	8.73%	127.29	5.3	5.33%
	3	Opti	130.43	4193.8	0.00%	–	–	–	130.43	2173.5	0.00%	130.43	113.0	0.00%	163.15	<0.1	25.09%	151.27	0.4	15.98%	144.37	2.5	10.69%
	4	Feas	116.26	–	3.53%	–	–	–	115.56	4556.7	0.70%	115.56	1718.1	0.67%	138.92	<0.1	20.22%	125.58	0.5	8.67%	124.70	3.5	7.91%
	5	Feas	137.00	–	4.19%	–	–	–	137.00	830.2	0.00%	137.00	31.0	0.00%	171.22	<0.1	24.98%	153.95	0.3	12.38%	153.25	4.9	11.86%
	6	Feas	134.95	–	1.19%	–	–	–	134.95	1551.5	1.82%	134.95	44.7	0.00%	165.88	<0.1	22.92%	153.27	0.2	13.58%	146.37	3.0	8.46%
	7	Feas	120.29	–	4.57%	–	–	–	120.29	4364.6	0.00%	120.29	1977.8	0.00%	142.92	<0.1	18.81%	132.48	0.3	10.13%	132.69	2.8	10.31%
	8	Feas	145.78	–	8.99%	–	–	–	–	–	–	144.38	5892.1	1.00%	176.04	<0.1	21.93%	154.75	0.4	7.18%	153.87	4.7	6.58%
	9	Feas	148.19	–	8.36%	–	–	–	–	–	–	–	–	–	175.05	<0.1	18.13%	152.29	0.6	2.77%	147.98	6.6	–0.14%
	10	Feas	94.09	–	9.04%	91.64	6051.5	2.94%	91.29	4283.0	5.64%	91.64	3925.9	2.67%	125.41	<0.1	36.85%	108.14	0.2	18.46%	108.14	1.4	18.46%
Avg /		142.22	>7082.7	9.37%	>144.4	>3605.6	>5.38%	>140.26	>1893.3	>2.93%	>141.13	>1314.1	>1.88%	173.06	<0.1	>22.81%	157.17	0.2	>11.40%	156.38	2.0	>10.80%	

–: for these instances, the optimal (feasible) solutions cannot be obtained within 7200 s, or the values are not available.

Table 10

Comparison of performance between Algorithms GreedyPack, CBD, two SA-based algorithms and the MILP model with $n = 40$.

Comb	Ins	MILP				Algorithm CBD2			Algorithm CBD3			Algorithm CBD4			Algorithm GreedyPack			Algorithm SA_FF			Algorithm SA_BF		
		Opt	C _{max}	Time	Gap ^o	C _{max}	Time	Gap ^o	C _{max}	Time	Gap ^o	C _{max}	Time	Gap ^o	C _{max}	Time	Gap ^f	C _{max}	Time	Gap ^f	C _{max}	Time	Gap ^f
10	1	-	-	-	-	245.62	498.4	0.00%	245.62	196.2	0.00%	245.62	204.2	0.00%	294.42	<0.1	19.87%	270.67	0.1	10.20%	270.67	0.3	10.20%
	2	-	-	-	-	241.01	1051.2	2.96%	241.01	808.1	1.87%	241.01	210.8	1.15%	293.31	<0.1	21.70%	260.31	0.1	8.01%	260.31	0.3	8.01%
	3	-	-	-	-	218.19	1718.7	6.14%	218.19	820.2	0.21%	218.19	606.1	0.21%	268.35	<0.1	22.99%	244.36	0.1	11.99%	244.18	0.3	11.91%
	4	-	-	-	-	230.56	1508.8	4.51%	230.56	473.5	0.10%	230.56	287.6	1.35%	281.67	<0.1	22.17%	256.87	0.1	11.41%	256.87	0.3	11.41%
	5	-	-	-	-	227.12	1019.8	0.00%	227.12	433.9	0.00%	227.12	201.5	0.00%	268.04	<0.1	18.02%	247.62	0.1	9.03%	247.62	0.3	9.03%
	6	-	-	-	-	225.60	740.7	0.00%	225.60	347.7	0.00%	225.60	117.9	0.00%	274.47	<0.1	21.66%	246.86	0.1	9.43%	253.80	0.3	12.50%
	7	-	-	-	-	214.61	759.6	0.00%	214.61	349.9	0.00%	214.61	326.6	0.00%	266.11	<0.1	24.00%	240.82	0.1	12.21%	238.12	0.3	10.95%
	8	-	-	-	-	234.21	916.2	0.00%	234.21	568.2	0.00%	234.21	199.2	1.79%	282.03	<0.1	20.42%	255.24	0.1	8.98%	255.24	0.3	8.98%
	9	-	-	-	-	226.37	655.0	0.00%	226.37	266.1	0.00%	226.37	67.9	0.00%	276.99	<0.1	22.36%	255.87	0.1	13.03%	255.87	0.3	13.03%
	10	-	-	-	-	229.33	3543.6	7.48%	229.33	2924.8	7.48%	229.33	2901.7	0.00%	283.97	<0.1	23.83%	260.12	0.1	13.43%	260.12	0.5	13.43%
11	1	-	-	-	-	209.23	5193.4	14.66%	209.93	5635.8	14.96%	211.33	4737.9	15.82%	243.99	<0.1	15.45%	222.65	0.2	6.41%	225.35	0.6	7.70%
	2	-	-	-	-	204.47	6385.7	19.98%	203.07	2793.6	17.03%	204.23	3557.4	19.70%	247.32	<0.1	21.10%	223.67	0.2	10.14%	225.07	0.5	10.83%
	3	-	-	-	-	187.64	3807.0	15.18%	185.30	2504.0	15.44%	185.30	1852.6	15.44%	226.14	<0.1	22.04%	204.50	0.1	10.36%	205.20	0.8	10.74%
	4	-	-	-	-	200.47	6128.7	17.23%	200.23	2691.0	17.11%	201.63	2561.5	17.98%	241.07	<0.1	19.56%	218.03	0.2	8.89%	217.75	0.6	8.75%
	5	-	-	-	-	197.88	5924.6	13.82%	195.78	3678.5	12.97%	195.08	3077.8	12.33%	230.99	<0.1	18.41%	208.95	0.1	7.11%	207.38	0.7	6.31%
	6	-	-	-	-	197.36	5719.3	14.72%	197.36	5738.6	14.78%	195.96	5258.4	14.19%	237.27	<0.1	21.08%	212.74	0.2	8.56%	212.74	0.6	8.56%
	7	-	-	-	-	-	-	-	-	-	-	-	-	-	216.02	<0.1	-	192.59	0.2	-	192.59	0.5	-
	8	-	-	-	-	198.98	6471.5	18.58%	198.98	3753.6	19.02%	198.98	3382.3	18.98%	234.92	<0.1	18.06%	217.83	0.1	9.47%	217.83	0.9	9.47%
	9	-	-	-	-	-	-	-	-	-	-	-	-	-	228.13	<0.1	-	209.64	0.2	-	209.99	0.6	-
	10	-	-	-	-	-	-	-	-	-	-	-	-	-	241.63	<0.1	-	227.54	0.2	-	226.74	0.7	-
12	1	-	-	-	-	-	-	-	185.73	4761.4	6.57%	184.33	1956.8	4.85%	231.80	<0.1	25.75%	198.03	0.3	7.43%	197.54	1.2	7.17%
	2	-	-	-	-	-	-	-	184.47	6770.4	14.35%	180.27	3564.7	8.87%	225.64	<0.1	25.16%	200.87	0.2	11.43%	200.87	1.4	11.43%
	3	Feas	176.30	-	22.30%	-	-	-	174.30	5322.6	9.78%	172.20	1363.1	5.00%	211.82	<0.1	23.01%	192.10	0.3	11.56%	194.20	1.2	12.78%
	4	-	-	-	-	-	-	-	185.03	4905.2	7.42%	185.03	1071.3	5.83%	224.19	<0.1	21.17%	208.43	0.2	12.65%	200.83	1.3	8.54%
	5	Feas	188.95	-	28.32%	-	-	-	174.83	2633.4	0.80%	174.83	325.1	2.11%	218.55	<0.1	25.01%	192.18	0.2	9.92%	193.05	1.4	10.42%
	6	Feas	183.46	-	25.10%	-	-	-	174.56	2594.9	4.16%	174.56	829.4	4.93%	219.33	<0.1	25.65%	194.85	0.1	11.62%	191.97	1.0	9.97%
	7	Feas	178.58	-	30.82%	-	-	-	164.88	2527.7	3.97%	164.88	521.5	3.97%	203.34	<0.1	23.33%	183.38	0.2	11.22%	180.68	1.0	9.58%
	8	Feas	187.78	-	23.24%	-	-	-	181.68	3408.8	6.30%	181.68	817.8	5.91%	227.04	<0.1	24.97%	201.58	0.2	10.95%	201.58	1.0	10.95%
	9	Feas	177.94	-	21.04%	-	-	-	171.14	6339.3	9.08%	171.14	1833.0	6.22%	213.70	<0.1	24.87%	197.94	0.3	15.66%	198.99	1.0	16.27%
	10	Feas	189.93	-	17.92%	-	-	-	181.73	5525.5	10.36%	181.73	697.9	1.77%	225.48	<0.1	24.08%	207.13	0.2	13.98%	207.13	1.0	13.98%
Avg		/	>183.28	>200.0	>24.11%	>216.98	>4854.8	>7.96%	>202.28	>3345.8	>7.18%	>202.07	>2137.7	>6.26%	244.59	<0.1	>22.06%	221.78	0.2	>10.56%	221.68	0.7	>10.48%

Table 11
Comparison of performance between Algorithms GreedyPack, CBD and two SA-based algorithms for combinations 1–6.

Combination	Algorithm CBD2		Algorithm CBD3		Algorithm CBD4		Algorithm GreedyPack			Algorithm SA_FF			Algorithm SA_BF		
	$C_{\max}$	Time	$C_{\max}$	Time	$C_{\max}$	Time	$C_{\max}$	Time	Gap ^o	$C_{\max}$	Time	Gap ^o	$C_{\max}$	Time	Gap ^o
1	77.08	9.7	77.08	7.9	77.08	9.6	92.23	<0.1	20.37%	83.14	0.3	7.95%	84.20	1.8	9.42%
2	68.29	72.3	68.29	79.6	68.29	96.2	81.33	<0.1	19.29%	73.77	0.6	8.21%	73.37	6.4	7.29%
3	66.18	5.2	66.18	10.4	66.18	10.1	74.99	<0.1	13.05%	68.79	1.4	4.05%	69.81	7.4	5.55%
4	108.75	110.1	108.75	37.5	108.75	40.0	130.69	<0.1	20.80%	117.49	0.2	8.49%	117.57	0.8	8.40%
5	94.65	362.8	94.65	124.1	94.65	142.1	114.34	<0.1	20.91%	103.88	0.4	9.60%	105.07	2.0	10.72%
6	87.17	256.7	87.17	88.6	87.17	90.5	106.03	<0.1	21.55%	95.77	0.6	9.99%	95.98	4.2	10.18%
Avg	83.69	136.1	83.69	58.0	83.69	64.7	99.94	<0.1	19.33%	90.47	0.6	8.05%	91.00	3.8	8.59%

of about 23%, and the computational time is minimal, only took <0.1 s of running time for each combination of the test instances in our computational experiments. Meanwhile, the SA-based algorithms proposed by Che et al. (2021) can also be implemented very quickly - with an average running time that is generally less than 2 s for each combination, and the average gaps of these two SA-based algorithms are better than those of Algorithm GreedyPack. Therefore, when we only have very limited computational time or the size of the instances is large, Algorithm GreedyPack and the two SA-based algorithms might be acceptable or even good alternatives to solve the SMSAM-2DP problem.

7. Conclusions

In this paper, we study a single-machine scheduling problem with two-dimensional packing constraints in additive manufacturing, i.e., the SAMSM-2DP problem, and the objective is to minimize the makespan. We first propose a mixed integer linear programming formulation for this problem, and design an approximation algorithm with a constant approximation ratio. Then, we develop various combinatorial Benders decomposition-based solution approaches with different types of acceleration strategies and Benders cuts. To evaluate the efficiency of our proposed solution approaches, we conduct extensive computational experiments on randomly generated instances. The computational results show that our proposed combinatorial Benders decomposition algorithms could be quite effective in finding the optimal solutions to the SAMSM-2DP problem within reasonable CPU running times.

Our work can be further extended to other possible research directions in the future. For example, it may be interesting to consider similar scheduling problems with parallel additive machines or with different manufacturing configurations, such as permitting orientation selection for each part. Second, we may incorporate other important objectives into the model, such as some other time-related or energy-related objectives. It would also be interesting to investigate the structural properties of the optimal solutions for the SAMSM-2DP problem, and develop other strengthened Benders cuts to accelerate the solution process of the problem.

CRediT authorship contribution statement

Zhaofang Mao: Conceptualization, Supervision. **Enyuan Fu:** Methodology, Software, Writing – original draft, Writing – review & editing, Data curation. **Dian Huang:** Resources, Validation. **Kan Fang:** Investigation, Methodology, Writing – review & editing, Validation. **Lin Chen:** Formal analysis, Methodology.

Acknowledgments

The authors are grateful to Michael L. Pinedo and three anonymous referees for their constructive comments which contributed to improve the quality of this paper. This work was supported by the National Natural Science Foundation Council of China under Projects 92167206, 71872125 and 72231005, and the Ministry of Science and Technology of China under Project 2020IM030300. This work was also

supported by Tianjin Philosophy and Social Science Planning Project, China (TJGL21-016).

References

Aloui, A., & Hadj-Hamou, K. (2021). A heuristic approach for a scheduling problem in additive manufacturing under technological constraints. *Computers & Industrial Engineering*, 154, Article 107115.

Altekin, F. T., & Bukchin, Y. (2022). A multi-objective optimization approach for exploring the cost and makespan trade-off in additive manufacturing. *European Journal of Operational Research*, 301(1), 235–253.

Attaran, M. (2017). The rise of 3-D printing: The advantages of additive manufacturing over traditional manufacturing. *Business Horizons*, 60(5), 677–688.

Benders, J. (1962). Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik*, 4, 238–252.

Boschetti, M. A., & Mingozzi, A. (2003a). The two-dimensional finite bin packing problem. Part I: New lower bounds for the oriented case. *4OR. A Quarterly Journal of Operations Research*, 1, 27–42.

Boschetti, M. A., & Mingozzi, A. (2003b). The two-dimensional finite bin packing problem. Part II: New lower and upper bounds. *4OR. A Quarterly Journal of Operations Research*, 1, 135–147.

Boyar, J., Dósa, G., & Epstein, L. (2012). On the absolute approximation ratio for First Fit and related results. *Discrete Applied Mathematics*, 160(13), 1914–1923.

Caprara, A., & Monaci, M. (2004). On the two-dimensional knapsack problem. *Operations Research Letters*, 32(1), 5–14.

Che, Y., Hu, K., Zhang, Z., & Lim, A. (2021). Machine scheduling with orientation selection and two-dimensional packing for additive manufacturing. *Computers & Operations Research*, 130, Article 105245.

Chung, F. R., Garey, M. R., & Johnson, D. S. (1982). On packing two-dimensional bins. *SIAM Journal on Algebraic Discrete Methods*, 3(1), 66–76.

Clautiaux, F., Carlier, J., & Moukrim, A. (2007). A new exact method for the two-dimensional orthogonal packing problem. *European Journal of Operational Research*, 183(3), 1196–1211.

Codato, G., & Fischetti, M. (2006). Combinatorial Benders' cuts for mixed-integer linear programming. *Operations Research*, 54(4), 756–766.

Cortés, C. E., Matamala, M., & Contardo, C. (2010). The pickup and delivery problem with transfers: Formulation and a branch-and-cut solution method. *European Journal of Operational Research*, 200(3), 711–724.

Côté, J.-F., Haouari, M., & Iori, M. (2021). Combinatorial Benders decomposition for the two-dimensional bin packing problem. *INFORMS Journal on Computing*, 33(3), 963–978.

Epstein, L. (2010). Two-dimensional online bin packing with rotation. *Theoretical Computer Science*, 411(31–33), 2899–2911.

Fang, K., Wang, S., Pinedo, M. L., Chen, L., & Chu, F. (2021). A combinatorial Benders decomposition algorithm for parallel machine scheduling with working-time restrictions. *European Journal of Operational Research*, 291(1), 128–146.

Gardan, J. (2016). Additive manufacturing technologies: State of the art and trends. *International Journal of Production Research*, 54(10), 3118–3132.

Grand View Research (2022). Additive manufacturing market size, share & trends analysis report by component, by printer type, by technology, by software, by application, by vertical, by material, by region, and segment forecasts, 2022–2030. <https://www.grandviewresearch.com/industry-analysis/additive-manufacturing-market#>. (Online; Accessed May 1 2024).

Guo, W., Jiang, P., & Yang, M. (2023). Unequal area facility layout problem-solving: a real case study on an air-conditioner production shop floor. *International Journal of Production Research*, 61(5), 1479–1496.

Heragu, S. S. (2022). *Facilities Design* (5th ed.). CRC Press.

Hooker, J. N. (2007). Planning and scheduling by logic-based Benders decomposition. *Operations Research*, 55(3), 588–602.

Hooker, J. (2011). *Logic-based methods for optimization: Combining optimization and constraint satisfaction*. John Wiley & Sons.

Hu, K., Che, Y., & Zhang, Z. (2022). Scheduling unrelated additive manufacturing machines with practical constraints. *Computers & Operations Research*, 144, Article 105847.

- Huang, D., Mao, Z., Fang, K., & Yuan, B. (2022). Combinatorial Benders decomposition for mixed-model two-sided assembly line balancing problem. *International Journal of Production Research*, 60(8), 2598–2624.
- Júnior, A. N., Silva, E., Francescato, M., Rosa, C. B., & Siluk, J. (2022). The rectangular two-dimensional strip packing problem real-life practical constraints: A bibliometric overview. *Computers & Operations Research*, 137, Article 105521.
- Kenmochi, M., Imamichi, T., Nonobe, K., Yagiura, M., & Nagamochi, H. (2009). Exact algorithms for the two-dimensional strip packing problem with and without rotations. *European Journal of Operational Research*, 198(1), 73–83.
- Kucukkoc, I. (2019). MILP models to minimise makespan in additive manufacturing machine scheduling problems. *Computers & Operations Research*, 105, 58–67.
- Kucukkoc, I. (2021). Metal additive manufacturing: Nesting vs. scheduling. In *AIRO springer series, Optimization and data science: trends and applications* (pp. 169–180). Springer.
- Li, Y., Côté, J.-F., Callegari-Coelho, L., & Wu, P. (2022). Novel formulations and logic-based Benders decomposition for the integrated parallel machine scheduling and location problem. *INFORMS Journal on Computing*, 34(2), 1048–1069.
- Li, Q., Kucukkoc, I., & Zhang, D. Z. (2017). Production planning in additive manufacturing and 3D printing. *Computers & Operations Research*, 83, 157–172.
- Lodi, A., Martello, S., & Monaci, M. (2002). Two-dimensional packing problems: A survey. *European Journal of Operational Research*, 141(2), 241–252.
- Oh, Y., Witherell, P., Lu, Y., & Sprock, T. (2020). Nesting and scheduling problems for additive manufacturing: A taxonomy and review. *Additive Manufacturing*, 36, Article 101492.
- Powell, B. (2021). *3D printers: Technical report*, <https://www.dimensions.com/collection/3d-printers>.
- Rahmaniani, R., Crainic, T. G., Gendreau, M., & Rei, W. (2017). The Benders decomposition algorithm: A literature review. *European Journal of Operational Research*, 259(3), 801–817.
- Steinberg, A. (1997). A strip-packing algorithm with absolute performance bound 2. *SIAM Journal on Computing*, 26(2), 401–409.
- Verstichel, J., Kinable, J., De Causmaecker, P., & Berghe, G. V. (2015). A Combinatorial Benders' decomposition for the lock scheduling problem. *Computers & Operations Research*, 54, 117–128.
- Wang, J.-Q., Fan, G.-Q., & Liu, Z. (2020). Mixed batch scheduling on identical machines. *Journal of Scheduling*, 23, 487–496.
- Westerweel, B., Basten, R. J., & van Houtum, G.-J. (2018). Traditional or additive manufacturing? Assessing component design options through lifecycle cost analysis. *European Journal of Operational Research*, 270(2), 570–585.
- Zhang, J., Yao, X., & Li, Y. (2020). Improved evolutionary algorithm for parallel batch processing machine scheduling in additive manufacturing. *International Journal of Production Research*, 58(8), 2263–2282.
- Zohali, H., Naderi, B., & Roshanaei, V. (2022). Solving the type-2 assembly line balancing with setups using logic-based Benders decomposition. *INFORMS Journal on Computing*, 34(1), 315–332.